

"Schola Europa Akadémia" Technikum, Gimnázium és Alapfokú Művészeti Iskola a Magyarországi Metodista Egyház fenntartásában



SZOFTVERFEJLESZTŐ ÉS -TESZTELŐ

5 0613 12 03

Dokumentáció

Készítette:

Baranyi Norbert 14/B

Csontos Kincső Anasztázia 14/A

Szekeres Levente 14/A

2026

0. Dokumentum adatai

1. Bevezetés

- 1.1. A Projekt Célja
- 1.2. Főbb Funkciók
- 1.3. Technológiai Stack
- 1.4. Fogalomtár (Glossary)

2. Rendszerarchitektúra

- 2.1. Magas Szintű Architektúra
- 2.2. Komponensek
- 2.3. Adatbázis Séma

3. Frontend Architektúra

- 3.1. Komponens Hierarchia
- 3.2. Állapotkezelés
- 3.3. Routing
- 3.4. UI/UX Design

4. Backend Architektúra

- 4.1. Backend funkciók és felelősségek
 - 4.1.1. Felhasználókezelés és autentikáció
 - 4.1.2. Jogosultságok és szerepkörök
 - 4.1.3. Jegyzetek kezelése
 - 4.1.4. Fájlfeltöltés és validáció
 - 4.1.5. Közösségi funkciók
 - 4.1.6. Jelentés és moderáció
 - 4.1.7. Profil és testreszabás
 - 4.1.8. Üzenetek, barátok, értesítések
 - 4.1.9. Csoport funkciók
 - 4.1.10. Lokalizáció
 - 4.1.11. Adatbázis hozzáférési segédek és biztonság
- 4.2. Adatbázis kapcsolat
- 4.3. Fájlkezelés

5. Deployment

- 5.1. Környezetek
- 5.2. Fejlesztői környezet
- 5.3. Kód commit és push
- 5.4. Code review
- 5.5. Hibaelhárítás

6. Biztonság

- 6.1. Autentikáció

- 6.2. Jogosultságkezelés (RBAC)

7. Tesztelés

- 7.1. Egység Tesztek
- 7.2. Manuális Tesztek

8. Felhasználói Dokumentáció

- 8.1. Elérés / Használatba vétel
- 8.2. Használat
 - 8.2.1. Regisztráció
 - 8.2.2. Bejelentkezés + 2FA
 - 8.2.3. Jegyzet keresése és letöltése
 - 8.2.4. Jegyzet feltöltése
 - 8.2.5. Kommentelés, értékelés, kedvencek
 - 8.2.6. Profilkezelés
 - 8.2.7. Barátok hozzáadása és üzenetek
- 8.3. Weben belüli navigáció (Oldaltérkép)
- 8.4. Biztonsági Tippet
- 8.5. Gyakori problémák (FAQ)

9. Fejlesztői Dokumentáció

- 9.1. Fejlesztői Környezet Beállítása
- 9.2. Verziókezelési Stratégia
 - 9.2.1. Verziószám felépítése
 - 9.2.2. Fejlesztési (beta) állapot jelölése
 - 9.2.3. Átmenet végleges verzióra
 - 9.2.4. Verziózás a CHANGELOG-ban
 - 9.2.5. Dátumformátum szabályok
 - 9.2.6. Verziókezelés és Git kapcsolata
 - 9.2.7. Összefoglalás
- 9.3. Fájlszerkezet
- 9.4. Fejlesztői eszközök, script-ek és refaktorok
- 9.5. Konfiguráció
- 9.6. Fő folyamatok
 - 9.6.1. 2FA
 - 9.6.2. Admin Panel
 - 9.6.3. Tanuló Csoport Létrehozása
 - 9.6.4. Kedvencek
 - 9.6.5. Elfelejtett jelszó
 - 9.6.6. Tanuló csoport
 - 9.6.7. Tanuló csoportok
 - 9.6.8. Főoldal
 - 9.6.9. Feltöltés
 - 9.6.10. Kereső
 - 9.6.11. Jegyzet
 - 9.6.12. Jegyzet Statisztikák

- 9.6.13. [Profil](#)
- 9.6.14. [Üzenetek](#)
- 9.6.15. [Értesítések](#)
- 9.6.16. [Bejelentkezés & Regisztráció](#)
- 9.6.17. [Email-s visszaigazolás](#)
- 9.6.18. [Adatvédelmi Tájékoztató](#)
- 9.7. [Security checklist \(dev\)](#)
- 9.8. [Debug / logging](#)
- 9.9. [Változásnapló \(Changelog\)](#)
- 9.10. [Dokumentáció karbantartás](#)

10. [API Dokumentáció](#)

11. [Jövőbeli Tervek](#)

- 11.1. [Kollaboratív tanulási eszközök](#)
- 11.2. [API és harmadik fél integrációk](#)
- 11.3. [Big data analitika](#)
- 11.4. [AI-alapú keresés és javaslatok](#)
- 11.5. [Gamifikáció](#)
- 11.6. [Mobil Applikáció](#)
- 11.7. [Interaktív tesztek](#)

12. [Ki mit készített?](#)

- 12.1. [Baranyi Norbert](#)
- 12.2. [Csontos Kincső Anasztázia](#)
- 12.3. [Szekeres Levente](#)
- 12.4. [Szekeres Levente & Baranyi Norbert](#)

13. [Licenz](#)

Dokumentum adatai

Tulajdonság	Érték
Projekt neve	Jegyzetár
Dokumentum típusa	Fejlesztői + felhasználói dokumentáció
Dokumentum azonosító	JEGYZETAR-DOKU-2026-01
Verzió	1.0.0
Kiadás dátuma	TBD
Utolsó frissítés	2026-01-15
Állapot	Draft
Célközönség	Diákok, tanárok, fejlesztők
Repository	doomhyena/jegyzetar.eu-src
Célplatform	Web (reszponzív)
Min. PHP	8.2+
Támogatott böngészők	Chrome, Firefox, Edge (friss)
Kapcsolat	info@jegyzetar.hu

Verziózás: SemVer (major.minor.patch)

Megjegyzés: A dokumentáció a projekt aktuális állapotát tükrözi, a változásokat a CHANGELOG rögzíti.

1. Bevezetés

1.1. A Projekt Célja

A **Jegyzetár** egy modern, közösségi alapú, oktatássegítő platform, amely lehetővé teszi a diákok számára, hogy egyszerűen és biztonságosan megosszák egymással jegyzeteiket, segédanyagaikat és tanulást támogató dokumentumaikat. A projekt célja, hogy:

- **Kik használják?**
Diákok, tanárok és oktatók, akik megosztanák egymással az oktatási anyagokat.
- **Gyors és kényelmes fájlmegosztást biztosítson tanulóknak**
 - A felhasználók könnyedén feltölthetnek, kereshetnek és letölthetnek jegyzeteket
 - A platform reszponzív kialakítása biztosítja a mobilbarát használatot
 - Az egyszerű kezelőfelület csökkenti az informatikai tudás iránti igényt
- **Rendszerezett és kereshető tananyagbázist hozzon létre**
 - Tantárgyak, évfolyamok és dokumentumtípusok szerint kategorizál
 - Kulcsszavas keresés és szűrés segíti a gyors anyagkeresést
 - Előnézeti kép vagy rövid leírás segít a tartalom gyors azonosításában
- **Támogassa a közösségi tanulást**
 - A felhasználók értékelhetik, kommentelhetik a jegyzeteket
 - A rendszer kiemeli a legnépszerűbb vagy legjobbra értékelt anyagokat

1.2. Főbb Funkciók

A Jegyzetár rendszer az alábbi kulcsfunkciókat biztosítja:

Fájlfeltöltés és -kezelés

- Jegyzetek és segédanyagok feltöltése PDF, DOCX, MP4 formátumban
- Automatikus kategorizálás tantárgy és dokumentumtípus szerint
- Előnézeti kép generálása (PDF első oldal, képek)

Felhasználói felület

- Regisztráció és bejelentkezés (alap és "OAuth")
- Egyéni irányítópult a feltöltések, letöltések, értékelések követésére
- Mobilbarát és reszponzív dizájn

Keresés és közösségi funkciók

- Kulcsszavas kereső és szűrő (tantárgy, értékelés alapján)
- Kommentelés, csillagozás
- Legnépszerűbb jegyzetek és új feltöltések kiemelése

Adminisztráció és jogosultságok

- Admin felület a tartalmak, felhasználók és kategóriák kezeléséhez
- Feltöltések jelentése és moderálása
 - Admin eszközök kiterjesztése: Külön felület az egyedi profil CSS kérések kezelésére (jóváhagyás / elutasítás / archiválás), valamint a felhasználók és feltöltések részletes kezelése (törlés, szerkesztés, moderation logs).

Profil egyedi CSS kérések

Bevezetésre került a felhasználói CSS kérelmek kezelése, amely a profil felületén megadható, de csak admin jóváhagyása után érvényesül. Az előnézet a felhasználói élményt javítja és nem ment a szerverre automatikusan.

Admin jóváhagyási UI

Az admin panelben külön táblában listázhatók és jóváhagyhatók a CSS kérések; az elfogadott CSS archiválható és az előző állapot is visszaállítható.

Frontend preview & safe background rules

A preview kliens oldali (style tag injektálás), a SAFE_BG_RULE szabály megakadályozza a background képek kellemetlen ismétlődését, és a preview idején a jobboldali panel rejtésre kerül, hogy az előnézet ne rontsa el az oldalt.

Lokalizációs módosítások

A `t()` segédfüggvény és `lang.php` központi megoldással egy adatbázis-alapú fordítási rendszer lett bevezetve. A fordítások hiányzó kulcsait automatikusan lehet seed-elni. Fordítási kulcsok a `profiles`-hoz a 1543-as ID-tól készültek.

SQL dump & seed eszközök

Hozzáadtuk a `clean_translations.py`, `translations_clean.sql`, `replace_translations_in_dump.py`, `repair_translations.sql` eszközöket, amelyek a dump-ok importjának biztonságát növelik (duplikált bejegyzések eltávolítása, ON DUPLICATE használata, táblák import sorrendje). A `repair_translations.sql` script automatikus backup-ot készít, majd törli a duplikátumokat és létrehozza a megfelelő UNIQUE indexet.

Biztonsági javítások

A kód refaktorálásával `db_prepared()` használata javasolt a lekérdezésekhez, `require_once` és `function_exists` védelem bevezetésre került, a `Message()` segédfüggvény javítja az értesítések konzisztenciáját.

1.3. Technológiai Stack

A Jegyzetár fejlesztése során a következő technológiákat és eszközöket használtuk, amelyek mindegyike szabadon elérhető és bármilyen modern számítógépen telepíthető:

Frontend

- **HTML5, CSS3, JavaScript:** Az alapvető webes technológiák a felhasználói felület kialakításához.
- **Bootstrap:** Reszponzív és mobilbarát dizájn gyors fejlesztéséhez.
- **jQuery:** Egyszerű DOM-manipulációk és AJAX-hívások kezelésére.

Backend

- **PHP (8.2+):** A szerveroldali logika és API-k implementálásához.

Adatbázis

- **MySQL:** Relációs adatbázis-kezelő a felhasználói adatok, fájlok és egyéb információk tárolására.
- **phpMyAdmin:** Az adatbázis adminisztrációjához.

Verziókezelés

- **Git:** Verziókövetés és csapatmunka támogatása.
- **GitHub:** Távoli repository a kód tárolására és megosztására.

Egyéb eszközök

- **XAMPP:** Lokális fejlesztői környezet (Apache, MySQL, PHP) - ingyenesen letölthető bármely operációs rendszerre.
- **Visual Studio Code:** Kódszerkesztő a fejlesztéshez - ingyenes és cross-platform.
- **PHPStorm:** Integrált Fejlesztői Környezet a fejlesztéshez - fizetős verzió elérhető bármeely operációs rendszerre (opcionális).

Felhasznált hardverek

Baranyi Norbert

-

Csontos Kincső Anasztázia

- **Laptop:** Lenovo LOQ 15A
- **CPU:** AMD Ryzen 5 7235HS
- **GPU:** NVIDIA GeForce RTX 3050 6GB Laptop GPU
- **RAM:** 16GB DDR5 4800 MHz
- **SSD:** 512GB NVMe SSD
- **OS:** Windows 11 Pro 64-bit

Szekeres Levente

-

Hardver követelmények

A fejlesztési környezet bármilyen modern számítógépen futtatható. Minimális követelmények:

- **Operációs rendszer:** Windows 10/11, macOS 10.14+ vagy Linux (Ubuntu/Debian)
- **Processzor:** Modern többmagos CPU (pl. Intel i5, AMD Ryzen 5 vagy hasonló)
- **Memória:** Legalább 8GB RAM (ajánlott 16GB a jobb teljesítmény érdekében)
- **Tárhely:** Legalább 50GB szabad hely SSD-n a projekt fájlok és virtuális környezet számára
- **Grafikus kártya:** Integrált vagy dedikált GPU (nem kritikus, mivel webfejlesztésről van szó)

1.4. Fogalomtár (Glossary)

- **2FA:** Kétféle hitelesítés, belépéskor e-mailben kapott kóddal.
- **OAuth:** Külső szolgáltatóval (pl. Discord) történő bejelentkezés.
- **Tag:** Kulcsszó címke a jegyzetek gyorsabb kereséséhez.
- **Moderáció:** Admin felügyelet (jelentések kezelése, tartalom törlés/szerkesztés).
- **Seed:** Kezdő (teszt) adatok betöltése adatbázisba.
- **Dump:** Adatbázis kimentés (.sql) importálható formában.

2. Rendszerarchitektúra

2.1. Magas Szintű Architektúra

A Jegyzetár egy háromrétegű architektúrát követ:

1. **Prezentációs réteg:** A felhasználói felület, amely a frontend technológiákra épül.
2. **Alkalmazásréteg:** A backend logika, amely PHP segítségével valósul meg.
3. **Adatbázis réteg:** A MySQL adatbázis, amely az összes adatot tárolja.

2.2. Komponensek

- **Frontend:** A felhasználói interakciók kezelése és az adatok megjelenítése.
- **Backend:** A logika és az adatbázis műveletek végrehajtása.
- **Adatbázis:** A felhasználói adatok, fájlok és metaadatok tárolása.

2.3. Adatbázis Séma

Az alábbi ábra a Jegyzetár adatbázis fő tábláit és azok kapcsolatait mutatja be:

[Hamarosan]

Jogosultsági szintek (áttekintés)

- **Guest (vendég):** böngészés, keresés, nyilvános jegyzetek megtekintése/letöltése (ha engedélyezett).
- **User (felhasználó):** feltöltés, komment, értékelés, kedvencek, profil szerkesztés, barátok/üzenetek.
- **Admin:** moderáció (jelentések), felhasználók kezelése, tartalmak törlése/szerkesztése, CSS kérelmek jóváhagyása/elutasítása.

3. Frontend Architektúra

3.1. Komponens Hierarchia

3.2. Állapotkezelés

3.3. Routing

3.4. UI/UX Design

A Jegyzetár felhasználói felülete egyszerű és intuitív, a következő szempontokat figyelembe véve:

- **Reszponzív dizájn:** A platform minden eszközön jól használható.
- **Egyszerű navigáció:** Könnyen elérhető funkciók és tiszta menürendszer.
- **Konzisztens stílus:** Azonos színpaletta és tipográfia az egész alkalmazásban.

Frontend kiegészítések

- A profiloldalhoz egy kliensoldali CSS előnézeti funkció került be: a felhasználó `profile.php` oldalon megadhat, és előnézhet egyedi CSS beállításokat. A preview csak kliensoldali stílus-injektálást használ és nem menti el automatikusan az adatbázisba.
- Biztonsági frontend szabályok: a preview megakadályozza a kedvezőtlen elrendezés-befolyásolást (jobboldali oszlop eltakarása preview alatt), illetve van egy `SAFE_BG_RULE` ami megakadályozza a háttér pattern (tiling) fokozott felvillanását.
- Üres CSS beküldésének megelőzése: kliens-oldali validáció, toast üzenet és szerveroldali fallback letakarítás, visszaállítási jelzés a felhasználó számára.

4. Backend Architektúra

4.1. Backend funkciók

A Jegyzetár backendje PHP alapon biztosítja az alkalmazás üzleti logikáját, a jogosultságkezelést, az adatbázis-műveleteket, valamint a fájlkezelést és az e-mail alapú folyamatokat.

4.1.1. Felhasználókezelés és autentikáció

- **Regisztráció és belépés** (`reglog.php`)
- **E-mail aktiváció:** regisztráció után aktiváló link küldése és validálása (`mail-regver.php`, `reg-ver.php`, `tokens` tábla)
- **Kétlépcsős hitelesítés (2FA):** belépés után e-mail kód küldése és ellenőrzése (`mail-2fa.php`, `2fa.php`, `2fa_codes` tábla)
- **Vendég mód:** a főoldal böngészése bejelentkezés nélkül (korlátozott jogosultság)

4.1.2. Jogosultságok és szerepkörök

- Guest / User / Admin jogosultsági szintek alkalmazása
- Admin funkciók elérése és védelme (moderáció, felhasználókezelés, CSS kérések)

4.1.3. Jegyzetek kezelése (CRUD + metaadat)

- Jegyzetek listázása és részletek megjelenítése (`index.php`, `note.php`)
- Metaadatok kezelése (név, leírás, tárgy, tag-ek)

- Letöltések kiszolgálása és hozzáférés-ellenőrzés ([download.php](#))

4.1.4. Fájlfeltöltés és validáció

- Fájl feltöltés kezelése ([upload.php](#))
- Kiterjesztés / méret / (opcionálisan MIME) ellenőrzés
- Fájlnevékezelés és biztonsági védelem (pl. path traversal megelőzés)
- Feltöltött fájlok és metaadatok mentése adatbázisba

4.1.5. Közösségi funkciók

- **Kommentek** kezelése ([comments](#) tábla)
- **Értékelés** kezelése ([ratings](#) tábla, egyszeri értékelés logika)
- **Kedvencek**: mentés és lista ([favorites.php](#), [favorites](#) tábla)

4.1.6. Jelentés és moderáció (report rendszer)

- Jelentés beküldése és tárolása ([reports](#) tábla, [report.php](#))
- Admin oldali kezelés: státuszok (open/dismissed/resolved), kezelő rögzítése
- Moderációs műveletek: tartalom/tevékenység kezelése (törlés/szerkesztés, ha implementálva)

4.1.7. Profil és testreszabás (CSS kérelmek)

- Profiladatok kezelése (bio, profilkép, téma)
- **Egyedi CSS kérelem** tárolása és státuszkezelése ([user_custom_css_requests](#))
- Jóváhagyás után archiválás ([user_custom_css_archive](#)) és admin döntési folyamat

4.1.8. Üzenetek, barátok, értesítések

- Barátkérelmek és státuszok kezelése ([friends](#))
- Privát üzenetek kezelése ([messages](#))
- Rendszerértesítések ([notifys](#))

4.1.9. Csoport funkciók

- Csoport létrehozás és kezelés ([groups](#), [group_members](#))
- Csoporton belüli fájlok: feltöltés, jóváhagyás, moderáció ([group_files](#))
- Csoportok integrációja keresésbe / navigációba / értesítésekbe (ha így van megoldva)

4.1.10. Lokalizáció (i18n)

- Nyelvek kezelése ([languages](#))
- Fordítások adatbázisban ([translations](#))
- [t\(\)](#) / [lang.php](#) alapú fordítás betöltés és missing-key seeding támogatás

4.1.11. Adatbázis hozzáférési segédek és biztonság

- Egységes DB hozzáférés wrapper-ek: [db_prepared](#), [db_stmt](#), [db_query](#)

- Prepared statement alapú lekérdezések preferálása (SQL injection kockázat csökkentése)
- Include/duplikáció védelem (`require_once`, `function_exists`)
- Egységes felhasználói üzenetek: `Message()` helper

4.2. Adatbázis Kapcsolat

A PHP mysqli-t használjuk az adatbázis műveletek végrehajtására.

- Security & prepared statements: A kód nagy részét átdolgoztuk, hogy a `db_prepared($conn, $sql, $types, $params)` wrapper-t használjuk, amely a mysqli prepared statements használatát biztosítja. Ezzel jelentősen csökkent a kockázata az SQL injekcióknak, és egységesebbé vált a lekérdezések kezelése.

4.3. Fájlkezelés

A feltöltött fájlokat a szerveren tároljuk, és a fájlokhoz tartozó metaadatokat az adatbázisban rögzítjük.

5. Deployment

5.1. Környezetek

- **Fejlesztői környezet:** Lokális XAMPP szerver.
- **Éles környezet:** Apache szerver MySQL adatbázissal.

5.2. Fejlesztői környezet

A fejlesztéshez szükséges összes függőség telepítése Composer és npm segítségével történik.

5.3. Kód commit és push

A kódot Git segítségével kezeljük, és minden változtatást a GitHub repository-ba push-olunk.

5.4. Code review

Minden új funkciót pull request formájában integrálunk, amelyet code review előz meg.

5.5. Hibaelhárítás

A hibák nyomon követésére és kezelésére a GitHub Issues funkcióját használjuk.

6. Biztonság

6.1. Autentikáció

6.2. Jogosultságkezelés (RBAC)

7. Tesztelés

7.1. Egység Tesztek

7.2. Manuális Tesztek

8. Felhasználói Dokumentáció

8.1. Elérés / Használatba vétel

A **Jegyzetár** egy webes platform, ezért **felhasználóknak nincs szükség telepítésre**. Elég egy modern böngésző és internetkapcsolat.

Előfeltételek (felhasználóknak)

- Modern böngésző: Chrome / Edge / Firefox (ajánlott friss verzió)
- Stabil internetkapcsolat
- Érvényes e-mail cím (regisztrációhoz és 2FA-hoz)

Első belépés előtt

1. Nyisd meg a Jegyzetár weboldalt (a tanár / rendszergazda által megadott címen).
2. Ha még nincs fiókod, regisztrálj (lásd 7.2).
3. Ha a rendszer e-mail aktivációt használ, ellenőrizd a postafiókod és aktiváld a fiókot.

Megjegyzés: A fejlesztői / lokális futtatás (XAMPP, adatbázis import, stb.) nem része a felhasználói használatnak - ezek a **Fejlesztői Dokumentációban** találhatók.

8.2. Használat

Ebben a részben a leggyakoribb felhasználói feladatok vannak leírva.

8.2.1. Regisztráció

1. Navigálj a [reglog.php](#) oldalra.
2. Töltsd ki a szükséges mezőket:
 - vezetéknév, keresztnév
 - felhasználónév
 - e-mail cím
 - jelszó
 - biztonsági kérdés / válasz (ha van)
3. Kattints a **Regisztráció** gombra.
4. (Ha szükséges) aktiváld a fiókot e-mailből a megadott linken ([reg-ver.php](#) / aktivációs oldal).

Tippek:

- Használj olyan jelszót, amit más oldalon nem használasz.
- E-mail címet pontosan add meg, mert a 2FA kód is oda érkezik.

8.2.2. Bejelentkezés + 2FA

1. Navigálj a [reglog.php](#) oldalra.
2. Kattints a **Lépj be!** linkre.
3. Add meg a felhasználóneved és jelszavad.
4. Kattints a **Bejelentkezés** gombra.

5. Ha a rendszer 2FA-t kér:

- ellenőrizd az e-mail fiókod (spam/promóciók mappát is),
- írd be a kapott kódot a 2FA oldalon ([2fa.php](#)).

8.2.3. Jegyzet keresése és letöltése

1. Nyisd meg a főoldalt ([index.php](#)).
2. Használd a keresőt vagy a szűrőket:
 - kulcsszó / cím / leírás
 - tantárgy
 - tagek
 - értékelés (ha van)
3. Kattints a kiválasztott jegyzetre (részletek oldal: [note.php](#)).
4. A jegyzet adatlapján kattints a **Letöltés** gombra/linkre.

Tipp: Ha túl sok találat van, szűkíts tantárgyra vagy adj meg specifikusabb kulcsszót.

8.2.4. Jegyzet feltöltése

1. Navigálj az [upload.php](#) oldalra.
2. Add meg a fájl adatait:
 - fájl neve / cím
 - leírás (ajánlott)
 - tantárgy / kategória (ha van)
 - tagek (ha van)
3. Válaszd ki a feltöltendő fájlt (pl. PDF, DOCX, MP4 - a rendszer beállításaitól függően).
4. Küldd el az űrlapot a feltöltéshez.

Fontos:

- Csak olyan fájlt tölts fel, amit megoszthatsz (saját jegyzet, saját anyag).
- Ne tölts fel személyes adatokat tartalmazó dokumentumot (pl. lakcím, telefonszám, osztálynapló fotó).

8.2.5. Kommentelés, értékelés, kedvencek

- **Kommentelés:** A [note.php](#) oldalon a komment mezőben írhatasz hozzászólást.
- **Értékelés:** A jegyzetet csillaggal/pontszámmal értékelheted (ha a rendszer engedélyezi).
- **Kedvencek:** Jelöld kedvencnek a jegyzetet (ha van ilyen gomb), majd a kedvenceket a [favorites.php](#) oldalon éred el.

8.2.6. Profilkezelés

1. Navigálj a [profile.php](#) oldalra.
2. Itt általában a következőket tudod kezelni:
 - profilkép feltöltése
 - bemutatkozás (bio)
 - téma (theme) beállítása (ha elérhető)

- saját feltöltések és aktivitás áttekintése

Profil testreszabása - Egyedi CSS (kérelem + preview)

1. A profilbeállításoknál található egy **CSS kód** mező, ahova saját profil CSS-t adhatsz meg.
2. Használhatod a **preview** gombot, amely **kliens oldali élő előnézetet** biztosít.
 - A változás ilyenkor csak előnézet, és **nem mentődik automatikusan**.
3. A szerkesztett CSS beküldéséhez használd az **Egyedi CSS elküldése** gombot - ez a kérés admin jóváhagyásra kerül.
4. Az admin döntés után:
 - jóváhagyás esetén a CSS érvénybe lép,
 - elutasítás esetén nem aktiválódik.

Megkötések / javaslatok:

- Ne próbálj "láthatatlan" gombokat vagy félrevezető UI-t készíteni (pl. letöltés elrejtése).
- Ha az előnézet "szétcsúsztatja" az oldalt, kapcsolj ki a preview-t és javítsd a CSS-t.

8.2.7. Barátok hozzáadása és üzenetek

Barát hozzáadása:

1. Nyisd meg egy felhasználó profilját (**profile.php**) vagy keress rá a **search.php** oldalon.
2. Használd a barát hozzáadása gombot.
3. Várd meg, míg a másik fél elfogadja.

Üzenetküldés:

1. Navigálj a **messages.php** oldalra.
2. Válaszd ki, kinek szeretnél írni.
3. Írd meg az üzenetet és küldd el.

8.3. Weben belüli navigáció (Oldaltérkép)

- **Főoldal (index.php)**: jegyzetek listája, kiemelt / új tartalmak, kereső és szűrők.
- **Keresés (search.php)**: részletes keresés és szűrés (kulcsszó, tantárgy, tag, stb.).
- **Jegyzet adatlap (note.php)**: jegyzet részletek, letöltés, kommentek, értékelés, kedvencek.
- **Feltöltés (upload.php)**: új fájl feltöltése metaadatokkal.
- **Kedvencek (favorites.php)**: elmentett jegyzetek listája.
- **Profil (profile.php)**: profil szerkesztése, profilkép, bio, téma, egyedi CSS kérelem.
- **Üzenetek (messages.php)**: privát üzenetek.
- **Értesítések (notify.php)**: rendszer értesítések (barát státusz, admin döntés, stb.).
- **Csoportok (groups.php, group.php, create_group.php)**: csoportok listája, csoport részletek, létrehozás.

Megjegyzés: Az **admin panel (admin_panel.php)** csak admin jogosultsággal érhető el.

8.4. Biztonsági Tippek

8.4.1. Fiókbiztonság

- Használj erős jelszót (legalább 10-12 karakter, szám + nagybetű + speciális jel ajánlott).
- Ne add ki a belépési adataid senkinek.
- Ha van 2FA, mindig használd, és ellenőrizd a spam mappát is, ha nem jön meg a kód.

8.4.2. Adat- és tartalombiztonság

- Ne tölts fel személyes adatokat tartalmazó dokumentumot (pl. lakcím, telefonszám, diákigazolvány fotó).
- Ne tölts fel jogvédezt tartalmat (pl. teljes tankönyv PDF), csak saját jegyzetet / saját készítésű anyagot.
- Letöltés előtt nézd meg a jegyzet leírását és a feltöltő által megadott információkat.

8.4.3. Fájlkézelés és óvintézkedések

- Csak megbízható forrásból származó fájlt nyiss meg.
- Ha gyanús tartalmat találsz, használd a jelentés funkciót (ha elérhető).
- Ha nyilvános gépen vagy:
 - kijelentkezés kötelező,
 - ne mentsd el a jelszót a böngészőben.

8.4.4. Jogosultságok tudatos használata

- Ne adj hozzá ismeretleneket barátoknak.
- Csoportoknál figyelj a "public/private" jellegre, és hogy ki láthatja a megosztott fájlokat.

8.5. Gyakori problémák (FAQ)

Nem tudok belépni. Mit tegyek?

- Ellenőrizd a felhasználónevet és a jelszót.
- Próbáld meg a jelszó visszaállítást a [forgotpass.php](#) oldalon (ha elérhető).

Nem érkezik meg a 2FA kód e-mailben.

- Nézd meg a spam/promóciók mappát.
- Várj pár percet és próbáld újakérni (ha van ilyen opció).
- Ellenőrizd, hogy helyes e-mail címmel regisztráltál.

Nem engedi feltölteni a fájlt.

- Ellenőrizd, hogy engedélyezett-e a fájltípus (pl. PDF/DOCX/MP4).
- Lehet, hogy túl nagy a fájl (méretlimit).
- Próbáld átnevezni a fájlt (ékezet/extra karakterek nélkül), és próbáld újra.

Feltöltöttem a fájlt, de nem látom.

- Frissítsd az oldalt (Ctrl+F5).
- Nézd meg a profilodnál / saját feltöltéseknél (ha van ilyen lista).

- Ha van moderáció/jóváhagyás, lehet, hogy várni kell admin döntésre.

A CSS preview "szétcsúsztatja" az oldalt.

- Kapcsold ki a preview-t.
- Egyszerűsítsd a CSS-t és kerüld a globális szabályokat (pl. `body { ... }` erős módosítása).
- Az admin elutasíthatja a zavaró vagy félrevezető dizájnt.

Nem találok egy jegyzetet keresésben.

- Próbálj rövidebb kulcsszót vagy tag-et.
- Szűrők (tantárgy, tag, értékelés) visszaállítása után próbáld újra.
- Lehet, hogy a jegyzet törölve lett / privát csoportban van.

Gyanús vagy szabálytalan tartalmat láttam.

- Használd a jelentés funkciót (ha van).
- Ne töltsd le / ne oszd tovább, és jelezd egy adminnak vagy tanárnak.

Gyakori rendszerüzenetek (példák)

- **"Hibás felhasználónév vagy jelszó."** – ellenőrizd a beírt adatokat.
- **"2FA kód lejárt."** – kérj új kódot, és próbáld meg újra.
- **"Nem engedélyezett fájl típus."** – csak a megadott formátumok tölthetők fel.
- **"A fájl túl nagy."** – csökkentsd a méretet vagy oszd több részre.
- **"Nincs jogosultság."** – a funkció csak bejelentkezve / admin jogosultsággal érhető el.

9. Fejlesztői Dokumentáció

9.1. Fejlesztői Környezet Beállítása

Quickstart (5 perc)

1. Repo klónozás (XAMPP/htdocs alá)
2. DB import: `assets/sql/jegyzetar.sql` (vagy `jegyzetar_clean.sql`)
3. `assets/php/db.php` beállítása
4. Böngésző: `http://localhost/jegyzetar.eu-src/src/`

Gyakori hibák

- 500 error: hiányzó PHP extension / rossz include path
- DB connection error: rossz host/user/pass/dbname
- Duplikált fordítások: `repair_translations.sql` futtatása vagy clean dump használata

1. Klónozd le a projekt fájljait (pl. ``git clone https://github.com/doomhyena/jegyzetar.eu-src.git``) a helyi szerver gyökérkönyvtárába (pl. ``c:/xampp/htdocs/jegyzetar.eu-src``).

2. Importáld az adatbázist:

- Nyisd meg a phpMyAdmin-t.

- Importáld a `Jegyzetár.sql` fájlt az `assets/sql/` mappából.
3. Konfiguráld az adatbázis kapcsolatot:
- Nyisd meg a `db.php` fájlt.
 - Győződj meg róla, hogy az adatbázis hitelesítési adatok megfelelnek a helyi szerver beállításainak.
4. Indítsd el a helyi szerveret, és navigálj a `http://localhost/jegyzetar.eu-src/src/` címre a böngésződben.

9.2. Verziókezelési Stratégia

A Jegyzetár projekt verziókezelése a **Semantikus Verziózás (Semantic Versioning – SemVer)** elveit követi, figyelembe véve, hogy a projekt jelenleg **fejlesztési / béta állapotban** van.

9.2.1. Verziószám felépítése

A verziószám formátuma:

```
major.minor.patch
```

Példa: 1.0.3

Rész	Jelentés
major	Nagy, kompatibilitást törő változás
minor	Új funkciók hozzáadása (visszafelé kompatibilis)
patch	Hibajavítások, kisebb módosítások

9.2.2. Fejlesztési (beta) állapot jelölése

A projekt jelenleg **nem tekinthető végleges, éles kiadásnak**, ezért a verziószám **előzetes (beta) jelölést** használ.

Használt forma:

```
[1.X.X]
```

Ez a jelölés azt fejezi ki, hogy:

- a **major verzió (1)** már tervezetten rögzített,
- a **minor és patch értékek még változhatnak**,
- a rendszer funkciói aktív fejlesztés alatt állnak.

Megjegyzés:

A **[1.X.X]** forma **nem végleges kiadást jelöl**, hanem egy **folyamatosan fejlődő verziócsaládot**.

Ez különösen hasznos iskolai / projektmunka környezetben, ahol a végleges release később történik.

8.2.3. Átmenet végleges verzióra

A projekt akkor tekinthető **első hivatalos kiadásnak**, amikor:

- a fő funkciók stabilak,
- a dokumentáció teljes,
- nincs kritikus hiba.

Ekkor a verziószám például:

1.0.0

Ezt követően:

- **1.1.0** = új funkciók
- **1.0.1** = hibajavítás
- **2.0.0** = nagy, visszafelé nem kompatibilis változás

9.2.4. Verziózás a CHANGELOG-ban

A változásnapló ([docs/CHANGELOG.md](#)) **minden verzióváltást rögzít**.

Fejlesztési állapotban:

```
[1.X.X] - 2026-01-15
Added
• ...
Changed
• ...
Fixed
• ...
```

Végleges verziónál:

```
[1.0.0] - 2026-04-30
Added
• Első stabil kiadás
```

9.2.5. Dátumformátum szabályok

A dokumentációban és a CHANGELOG-ban **egységes dátumformátum** kerül használatra:

YYYY-MM-DD

Példa:

- 2026-01-15
- 2026-04-30

Ez a formátum:

- egyértelmű,
- nem nyelvfüggő,
- nem keverhető össze (pl. US/EU dátumformátumokkal).

9.2.6. Verziókezelés és Git kapcsolata

- A verziószám **logikai állapotot** jelöl (nem minden commit növeli).
- Verzióváltás akkor történik, ha:
 - nagyobb funkció bekerül,
 - release készül,
 - CHANGELOG frissül.

Git tagek ajánlott formátuma:

v1.0.0
v1.1.0

9.2.7. Összefoglalás

- A projekt jelenleg **beta / fejlesztési fázisban** van
- A **[1.X.X]** jelölés ezt tudatosan kommunikálja
- A verziózás a **SemVer szabályait követi**
- A végleges kiadás **1.0.0** verzióval történik
- A CHANGELOG és a dokumentáció **összhangban van**

9.3. FájlStruktúra

[illegible]

9.4. Fejlesztői eszközök, script-ek és refaktorok

- `assets/php/functions.php` - Központi helyre került a `db_prepared()` segédfüggvény, ami a mysqli prepared statements használatát segíti és csökkenti az SQL injection kockázatát. A fájlban `function_exists` ellenőrzésekkel védjük a többszörös deklarációt.
- Include és duplikációs védelmek: Az include/require helyeken mostantól `require_once` használata ajánlott, és a központi függvényeknél `function_exists`-es feltételek alkalmazása segít elkerülni fatal hibákat többszörös include esetén.
- `assets/php/lang.php` - Feltölti és betölti az adatbázisban tárolt fordításokat (i18n). Egy beépített rutin gondoskodik arról, hogy a hiányzó kulcsok bekerüljenek az adatbázisba a támogatott nyelvekhez, így a `t()` segédfüggvény mindig vissza tud adni fordítást a kulcshoz.
- `assets/js/script.js` - Új kliensoldali funkciók: a profil egyedi CSS előnézete, preview bekapcsolás/letiltás, jobb oszlop rejtése preview közben, és kliens oldali validálás (pl. üres CSS megelőzése).
- `assets/php/functions.php` - A fejlesztés során bevezetett `Message()` segédfüggvény a felhasználói üzenetek központi kiírására szolgál; a korábbi `echo "<script>alert('...')";` szerkezeteket érdemes ezzel kiváltani a konzisztens, könnyebben tesztelhető és lokalizálható felhasználói értesítésekhez.

Fejlesztői script-ek az `assets/sql/scripts/` mappában:

- `clean_translations.py` - Python script, ami a nagy `translations` INSERT tömbből kiszedi a duplikált (`t_key`, `lang_code`) bejegyzéseket, és előállít egy `translations_clean.sql` fájlt, amely csak egyedi fordításokat tartalmaz.
- `replace_translations_in_dump.py` - Segédfájl, amellyel a `jegyzetar.sql` dumpból lecseréljük a `translations` beszárási blokkot a `translations_clean.sql` tartalmára, és előállítjuk a `jegyzetar_clean.sql` fájlt.
- `repair_translations.sql` - SQL script, amely a már létező adatbázison belül végrehajtható módon:
 1. biztonsági mentést készít a `translations` tábláról (`translations_backup`),
 2. törli a duplikált bejegyzéseket (például megtartja a `MIN(id)` vagy `MAX(id)` bejegyzéseket),
 3. létrehozza a `UNIQUE` indexet `t_key, lang_code` mezőkre.

- Import javaslat: Ha gyakran importálnak db dump-ot, importálják először a séma CREATE TABLE részt, majd az **ALTER TABLE**-okat (kulcsokkal), és végül a seed adatokat (INSERT), vagy használják a **jegyzetar_clean.sql** fájlt amely előre eltávolítja a duplikátumokat.
- A **profile.php** oldalhoz kapcsolódó fordítási kulcsok (HU/EN/DE) seeding-je **ID = 1543**-tól készült el. Amennyiben a roll-out során seed fájlokat használunk, ezek az értékek biztosítják a következetes fordítási kulcstartományt.

9.5. Konfiguráció

- DB: **assets/php/db.php**
- Upload path: **src/users/** (jogosultságok!)
- OAuth: **assets/oauth/** + szükséges kulcsok (.env ha van)
- Mail: **assets/php/mail-*.php** (SMTP / sender beállítások)

9.6. Fő folyamatok

Egyedi CSS kérés folyamata

1. User kitölti a CSS mezőt + preview (kliensoldali)
2. Beküldés → **user_custom_css_requests** (pending)
3. Admin panel: approve/reject
4. Approved esetén archiválás → **user_custom_css_archive**

9.6.1. 2fa.php

Oldal neve: 2fa.php

Cél: A bejelentkezés második lépcsője: az e-mailben kapott egyszer használatos kód ellenőrzése, majd siker esetén beléptetés.

Elérés / route:

- URL: **.../2fa.php**
- Belépés szükséges? **User** (legalább **\$_SESSION['id']** és **\$_SESSION['email']** megléte kötelező)

Bemenetek (inputok):

- GET: nincs érdemi GET paraméter
- POST:
 - **code** (string): a felhasználó által beírt 2FA kód
- Session:
 - **id** (int): felhasználó azonosító
 - **email** (string): felhasználó email címe
 - **tries** (int): hibás próbálkozások száma (ha nincs, 0-ra inicializálódik)

Folyamat (lépések):

1. Biztonsági headerek beállítása

- `X-Frame-Options: DENY` (clickjacking ellen)
- `X-Content-Type-Options: nosniff`
- `Referrer-Policy: no-referrer`

2. Session előkészítés

- Ha `$_SESSION['tries']` nincs beállítva: 0 értékre állítja.

3. Jogosultság ellenőrzés

- Ha nincs `$_SESSION['id']` vagy `$_SESSION['email']`:

- `redirect` → `reglog.php`
- `exit`

4. Oldal megjelenítés (navbar + űrlap)

- Rendereli az oldalt és a kód bekérésére szolgáló formot.

5. Kód ellenőrzés (POST esetén)

- Ha érkezett `$_POST['code']`:

1. `trim()` → `$code`

2. Ha `$code` nem üres:

- DB lekérdezés: van-e találat a `2fa_codes` táblában a (`userid`, `code`) párosra:

- ```
SELECT id FROM 2fa_codes WHERE userid = ? AND code = ?
LIMIT 1
```

3. **Siker ág** (pontosan 1 találat):

- Törli a kódot (egyszer használatos):

- ```
DELETE FROM 2fa_codes WHERE userid = ? AND code = ?
```

- Beállít egy cookie-t:

- ```
setcookie("id", $userid, time() + 3600, "/");
```

 (1 óra)

- `session_destroy()` (a 2FA session vége)

- `redirect` → `index.php` + `exit`

4. **Hiba ág** (nincs találat / üres kód):

- JS alert: `Helytelen kód`

- `$_SESSION['tries']++`

- Ha `tries >= 3`:

- `session_destroy()`
- `redirect` → `reglog.php` + `exit`

### UI állapotok / felhasználói visszajelzés:

- Loading: nincs
- Empty state: nincs külön kezelve
- Error state:
  - Hibás kód esetén: `alert('Helytelen kód')`
  - 3 hibás próbálkozás után: visszadob a `reglog.php` oldalra (session törléssel)
- Success state:
  - Helyes kód esetén: kód törlése + cookie beállítás + redirect `index.php`

### Biztonság:

- SQL injection védelem: paraméterezett DB hívások (`db_query`, `db_exec` helyettesítőkkal)
- XSS védelem: itt nincs visszatükrözött user input HTML-be (alert fix szöveg), így közvetlen XSS felület nem látszik
- Brute force védelem: session alapú próbálkozás limit (`tries`, max 3)
- Clickjacking/Content sniffing/Referrer védelem: a fenti HTTP headerek
- CSRF token: **nincs** a formban (jelenleg nem implementált)

### Elfogadási kritériumok (tesztelhető):

- "Ha nincs bejelentkezési session (`id/email`), az oldal `reglog.php`-ra redirectel."
- "Helyes kód esetén a `2fa_codes` rekord törlődik, `id` cookie beállítódik 1 órára, majd `index.php` redirect történik."
- "Hibás kód esetén `tries` növekszik és a felhasználó figyelmeztetést kap."
- "3 egymást követő hibás kód után a session törlődik és `reglog.php` redirect történik."
- "Üres kódra nem történik DB törlés/'beléptetés', és hibának minősül."

Ha szeretnéd, ugyanebben a sablonban leírom a *2FA kód generálás/kiküldés* oldalát/funkcióját is (ahol a `2fa_codes` rekord létrejön és az email kimegy) - mert a teljes 2FA flow igazán ott válik kerekké.

### 9.6.2. Admin Panel

**Oldal neve:** `admin_panel.php`

**Cél:** Admin felület az oldal tartalmainak és moderációs elemeinek kezelésére:

felhasználók/fájlok/kommentek törlése, kategóriák "üritése", profil CSS kérelmek bírálata, badge-ek kezelése és kiosztása, jelentések kezelése, regisztrációs kódok kezelése.

#### Elérés / route:

- URL: `.../admin_panel.php`
- Belépés szükséges? **Admin**



- Cookie alapú azonosítás: `$_COOKIE['id']`
- Admin jogosultság: `users.admin == 1`

## Bemenetek (inputok):

- GET:
  - `delete_type` (string): `user|file|comment|category|user_badge|badge`
  - `delete_id` (int): törlendő rekord azonosítója (több típusnál)
  - `subject` (string): kategória törlésnél a kategória neve (URL-enkódolva érkezik)
  - `css_action` (string): `approve|reject`
  - `css_id` (int): CSS kérelem ID
- POST:
  - Badge hozzárendelés:
    - `action=add_user_badge`
    - `user_id` (int), `badge_id` (int)
  - Badge CRUD:
    - `badge_action=create|update`
    - create: `name`, `slug`, opcionálisan `description`, `icon`
    - update: `badge_id` + ugyanazok a mezők
  - Jelentések kezelése:
    - `report_action=resolve|dismiss`
    - `report_id` (int)
  - Regisztrációs kódok:
    - létrehozás: `create_reg_code` + `code`, `description`, `max_uses`, `expires_at`
    - aktiválás/deaktiválás/törlés: `activate_reg_code` / `deactivate_reg_code` / `delete_reg_code` + `reg_code_id`
- Session: a fájl nem sessiont használ auth-hoz, hanem **cookie**-t:
  - `id` cookie kötelező és numerikus

## Folyamat (lépések):

### 1. Biztonsági headerek beállítása

- `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer`

### 2. Azonosítás cookie alapján

- Ha nincs `$_COOKIE['id']` vagy nem numerikus (`ctype_digit`):
  - `redirect` → `reglog.php` + `exit`

- `userId = (int)$_COOKIE['id']`
- DB: `SELECT * FROM users WHERE id = ? LIMIT 1`
- Ha nincs ilyen user:
  - `redirect` → `reglog.php` + `exit`

### 3. Admin jogosultság ellenőrzés

- Ha `users.admin != 1`:
  - HTTP 403 + `exit('Hozzáférés megtagadva...')`
- (A body-ban van egy **második** ellenőrzés is: ott UI-s üzenetet renderel és leáll.)

### 4. Admin műveletek kezelése (POST/GET)

- **User badge hozzárendelés** (POST `action=add_user_badge`)
  1. `user_id`, `badge_id`, `adminId` integerre
  2. Ellenőrzi, létezik-e már ilyen sor (`user_badges`):
    - `SELECT id FROM user_badges WHERE user_id=? AND badge_id=? LIMIT 1`
  3. Ha nem létezik: `INSERT INTO user_badges (user_id, badge_id, granted_by) VALUES (?, ?, ?)`
  4. Kimenet: JS `redirect admin_panel.php` + `exit`
- **Badge létrehozás / szerkesztés** (POST `badge_action`)
  - `create`:
    - validál: `name` és `slug` nem üres
    - `description/icon` üresen → `NULL`
    - `INSERT INTO badges (...) VALUES (?, ?, ?, ?)`
  - `update`:
    - validál: `badge_id > 0`, `name` és `slug` nem üres
    - `UPDATE badges SET ... WHERE id=? LIMIT 1`
  - Kimenet: JS `redirect admin_panel.php` + `exit`
- **Törlések / kategória kezelés** (GET `delete_type`, `delete_id`)
  - `user`:
    - ha nem saját fiók:
      - `DELETE FROM users WHERE id=?`

- `DELETE FROM files WHERE uploaded_by=?`
    - `DELETE FROM comments WHERE userid=?`
  - `file:`
    - `DELETE FROM files WHERE id=?`
    - `DELETE FROM comments WHERE postid=?`
  - `comment:`
    - `DELETE FROM comments WHERE id=?`
  - `category:`
    - `subject` esetén: `UPDATE files SET subject='' WHERE subject=?`
    - (tehát nem a "kategória rekordot" törli, hanem kiüríti a fájlokból)
  - `user_badge: DELETE FROM user_badges WHERE id=?`
  - `badge: DELETE FROM badges WHERE id=?`
  - Kimenet: JS redirect `admin_panel.php` + exit
- **Profil CSS kérelmek bírálata** (GET `css_action`, `css_id`)
- `approve:`
    1. lekéri a kérést: `SELECT * FROM user_custom_css_requests WHERE id=? LIMIT 1`
    2. ha megvan:
      - adott sor: `status='approved', reviewed_at=NOW(), reviewed_by=adminId`
      - ugyanazon user többi pending kérelme: `status='rejected'` (kivéve a mostani)
  - `reject:`
    - adott sor: `status='rejected', reviewed_at=NOW(), reviewed_by=adminId`
  - Kimenet: JS redirect `admin_panel.php` + exit
- **Jelentések kezelése** (POST `report_action`, `report_id`)
- csak `resolve` vagy `dismiss`
  - DB: `UPDATE reports SET status=?, handled_by=?, handled_at=NOW() WHERE id=?`
  - Kimenet: JS redirect `admin_panel.php#reports` + exit
- **Regisztrációs kód létrehozása** (POST `create_reg_code`)
1. `code` trim, üres → alert és nem ír DB-t

2. `max_uses` ha szám → int, különben NULL
3. `expires_at` ha megadva: `datetime-local` formátum T cserével + `:00`
4. DB: `INSERT INTO reg_codes (code, description, max_uses, expires_at, active) VALUES (?, ?, ?, ?, 1)`
5. Kimenet: alert "létrehozva" (a page marad)

- **Regisztrációs kód aktiválás/deaktiválás/törlés (POST)**

- `UPDATE reg_codes SET active=0/1 WHERE id=?`
- `DELETE FROM reg_codes WHERE id=?`

## 5. Listák lekérdezése (renderhez)

- `reg_codes, users, files, comments + username join, categories DISTINCT subject, css_requests, user_badges` joinok, `badge_options, user_options, badges, reports` + cél típus szerinti extra lookup a táblákból (users/groups/files).

## 6. Kimenet (UI render)

- Táblázatos admin felület szekciókkal:
  - Felhasználók / Fájlok / Kommentek / Kategóriák / CSS kérelmek / User badge kiosztás+lista / Badge CRUD / Jelentések / Regisztrációs kódok

## UI állapotok / felhasználói visszajelzés:

- Loading: nincs
- Empty state:
  - Jelentések: "Nincs még jelentés."
  - Reg kódok: "Még nincs regisztrációs kód."
- Error state:
  - Nem admin: 403 "Hozzáférés megtagadva..." (vagy body-ban kártyaüzenet)
  - Reg kód létrehozásnál üres kód: `alert('A kód mező nem lehet üres.')`
- Success state:
  - Több művelet után JS redirect vissza az admin panelre
  - Reg kód létrehozásnál: `alert('Regisztrációs kód létrehozva.')`

## Biztonság:

- SQL injection védelem: jellemzően paraméterezett hívások (`db_query`, `db_exec`, `db_stmt`) használata
- XSS védelem: listázásoknál több helyen `htmlspecialchars()` (pl. username, email, subject, CSS, report reason)
- Jogosultság: admin ellenőrzés `users.admin == 1` alapján (cookie → user betöltés → admin flag)
- CSRF token: **nincs** (sem a POST formokban, sem a GET-es "törlés/jóváhagyás" műveleteknél)

- Destruktív műveletek GET-tel: törlés/jóváhagyás részben **GET paraméterekkel** történik, csak `confirm()` JS védelemmel (ez UX, nem biztonság)

### Elfogadási kritériumok (tesztelhető):

- "Cookie `id` nélkül vagy nem numerikus `id`-vel a rendszer `reglog.php`-ra redirectel."
- "Nem admin user 403-at kap és nem látja az admin panelt."
- "Saját felhasználó nem törölhető (UI-ban 'Saját fiók', backendben is tiltva van a user delete ágon)."
- "Badge hozzárendelésnél duplikált `user_badges` sor nem jön létre."
- "Jóváhagyott CSS kérelem után a felhasználó többi pending kérelme automatikusan rejected lesz."
- "Regisztrációs kód üres `code` mezővel nem kerül DB-be."
- "Report resolve/dismiss esetén a `reports.status`, `handled_by`, `handled_at` frissül, majd a felület a #reports szekcióra ugrik."

### 9.6.3. Tanuló csoport létrehozása

**Oldal neve:** `create_group.php`

**Cél:** Új tanulócsoport létrehozása. A bejelentkezett felhasználó megadja a csoport nevét/leírását, opcionálisan privátra állítja, majd a rendszer létrehozza a `groups` rekordot és automatikusan belépteti a létrehozót "owner" szerepkörrel.

#### Elérés / route:

- URL: `.../create_group.php`
- Belépés szükséges? **User** (cookie alapú azonosítás: `$_COOKIE['id']`)

#### Bemenetek (inputok):

- GET: nincs érdemi GET paraméter
- POST (csak ha `letrehozás` be van küldve):
  - `letrehozás` (submit jelző)
  - `name` (string): csoport neve
  - `description` (string): csoport leírása (opcionális)
  - `is_private` (checkbox): ha be van pipálva → privát csoport
- Cookie:
  - `id` (string/int): bejelentkezett felhasználó azonosítója (csak numerikus fogadható el)

#### Folyamat (lépések):

##### 1. Biztonsági headerek beállítása

- `X-Frame-Options: DENY`
- `X-Content-Type-Options: nosniff`
- `Referrer-Policy: no-referrer`

##### 2. Jogosultság ellenőrzés

- Ha nincs `$_COOKIE['id']` vagy nem numerikus (`ctype_digit`):

- `redirect` → `reglog.php`
- `exit`

### 3. Input beolvasás és validálás (POST esetén)

- Csak akkor fut, ha `isset($_POST['letrehozas'])`
- `name` és `description` trimelve:
  - `$csoport_nev = trim($_POST['name'] ?? '')`
  - `$csoport_leiras = trim($_POST['description'] ?? '')`
- Privát flag:
  - `$privat = isset($_POST['is_private']) ? 1 : 0`
- Tulajdonos:
  - `$tulaj_id = (int)$_COOKIE['id']`
- Validálás:
  - Ha a csoport neve üres:
    - `alert('A csoport neve kötelező!')`
    - nincs DB írás

### 4. DB műveletek

- Csoport létrehozása:
  - `INSERT INTO groups (name, description, owner_id, is_private) VALUES (?, ?, ?, ?)`
- Ha az insert sikeres (`$inserted > 0`):
  1. `$uj_csoport_id = $conn->insert_id`
  2. Létrehozó felvétele tagként "owner" role-lal, elfogadott státusszal:
    - `INSERT INTO group_members (group_id, user_id, role, status) VALUES (?, ?, 'owner', 'accepted')`
  3. `alert('Csoport sikeresen létrehozva!')`
  4. `redirect` → `group.php?id=<új_id> + exit`
- Ha az insert nem sikerül:
  - `alert('Hiba történt a csoport létrehozásakor.')`

### 5. Kimenet (UI)

- Form mezők:
  - Csoport neve (input)
  - Leírás (textarea)
  - Privát csoport checkbox + magyarázó szöveg
- Gombok:
  - "Csoport létrehozása" (POST)
  - "Mégse, vissza a listához" → `groups.php`

### UI állapotok / felhasználói visszajelzés:

- Loading: nincs
- Empty state: nincs
- Error state:
  - Üres név: alert "A csoport neve kötelező!"
  - DB insert hiba: alert "Hiba történt a csoport létrehozásakor."
- Success state:
  - alert "Csoport sikeresen létrehozva!"
  - redirect a frissen létrehozott csoport oldalára: `group.php?id=...`

### Biztonság:

- SQL injection védelem: paraméterezett DB hívás (`db_exec` placeholder-ekkel)
- XSS védelem: ezen az oldalon a beküldött érték nincs visszarendelve (nincs `echo` a `name/description-ra`), így közvetlen XSS felület itt nem látszik
- CSRF token: **nincs** (a POST kérés nincs CSRF-fel védve)
- Auth: cookie `id` ellenőrzés (numerikus), de nem ellenőrzi itt külön, hogy a cookie-ban lévő user valóban létezik-e a `users` táblában (csak annyit, hogy szám)

### Elfogadási kritériumok (tesztelhető):

- "Nem bejelentkezett felhasználó (`id` cookie nélkül / nem numerikus `id`-vel) a `reglog.php` oldalra kerül."
- "Üres csoportnév esetén nem történik DB írás, és megjelenik a kötelező mező hibaüzenet."
- "Sikeres létrehozáskor létrejön egy rekord a `groups` táblában a helyes `owner_id` és `is_private` értékekkel."
- "Sikeres létrehozáskor létrejön egy rekord a `group_members` táblában `role='owner'` és `status='accepted'` értékekkel a létrehozó userre."
- "Siker esetén pontos redirect történik: `group.php?id=<új_csoport_id>.`"

### 9.6.4. Kedvencek

**Oldal neve:** `favorites.php`

**Cél:** A bejelentkezett felhasználó kedvencnek jelölt jegyzeteinek (fájlok) listázása és megjelenítése, elérés

biztosítása a részletekhez és letöltéshez.

### Elérés / route:

- URL: `.../favorites.php`
- Belépés szükséges? **User** (cookie alapú azonosítás: `$_COOKIE['id']`)

### Bemenetek (inputok):

- GET: nincs érdemi GET input
- POST: nincs (csak megjelenítő oldal)
- Cookie:
  - `id` (string/int): bejelentkezett felhasználó azonosítója (numerikus)
- Session: nincs használva ebben a fájlban

### Folyamat (lépések):

#### 1. Jogosultság ellenőrzés

- Ha nincs `$_COOKIE['id']` vagy nem numerikus (`ctype_digit`):
  - redirect → `reglog.php` + `exit`
- Betölti a user:
  - `SELECT * FROM users WHERE id = ? LIMIT 1`
- Ha nincs találat:
  - redirect → `reglog.php` + `exit`

#### 2. Input validálás

- Nincs klasszikus input validálás (nincs GET/POST), csak cookie `id` típusellenőrzés és user létezés ellenőrzés.

#### 3. DB művelet(ek)

- Értesítések száma (nem közvetlenül a kedvencekhez, de UI-hoz):
  - `SELECT id FROM notifys WHERE toid = ? AND readed = 0`
- Kedvencek lekérése:
  - `SELECT * FROM favorites WHERE user_id = ?`
- Minden kedvenc sorra:
  - `file_id` alapján fájl betöltése:
    - `SELECT * FROM files WHERE id = ? LIMIT 1`



- ha van találat: hozzáadja a `$favorites` tömbhöz
- Renderelés közben (minden listázott fájlnál):
  - feltöltő username lekérése:
    - `SELECT username FROM users WHERE id = ? LIMIT 1`
  - értékelések átlaga + darabszám:
    - `SELECT IFNULL(AVG(rating),0) as avg, COUNT(*) as c FROM ratings WHERE file_id = ?`

#### 4. Kimenet

- Siker: oldal renderelése a kedvenc fájlok kártyáival
- Üres lista: "Még nincsenek kedvenc jegyzeteid" empty state kártya
- Hiba (auth): redirect `reglog.php`

#### UI állapotok / felhasználói visszajelzés:

- Loading: nincs
- Empty state:
  - Ha `$favorites` üres: üzenet és ikon ("Még nincsenek kedvenc jegyzeteid")
- Error state:
  - Nincs bejelentkezés / érvénytelen user: redirect `reglog.php`
- Success state:
  - Kedvencek megjelenítése kártyákban:
    - cím (`name`)
    - feltöltő profil link (`profile.php?userid=...`)
    - "Részletek" link: `note.php?id=...`
    - "Letöltés" link: `assets/php/download.php?id=...`

#### Biztonság:

- SQL injection védelem: paraméterezett lekérdezések (`db_query`) használata
- XSS védelem:
  - Megjelenített dinamikus mezők escape-elve: `htmlspecialchars($f['name'])`, `htmlspecialchars($uploader['username'])`, stb.
- Fájl esetén:
  - A `favorites.php` nem tölt fel fájlt; letöltés külön endpointon (`download.php?id=...`) történik (annak a védelme ott releváns).

- CSRF token: nem releváns (nincs állapotmódosító POST ezen az oldalon)

### Elfogadási kritériumok (tesztelhető):

- "Cookie `id` nélkül vagy nem numerikus `id`-vel a felhasználó `reglog.php`-ra redirectelődik."
- "Ha a cookie alapján nem létező user azonosító jön, szintén `reglog.php`-ra redirectelődik."
- "Ha van legalább 1 kedvenc rekord, a hozzá tartozó fájl(ok) kártyái megjelennek."
- "Ha nincs kedvenc, az empty state ('Még nincsenek kedvenc jegyzeteid') jelenik meg."
- "A listában lévő elemeknél a fájlnev és username XSS-védetten jelenik meg (`htmlspecialchars`)."
- "A 'Részletek' link `note.php?id=<file_id>`-re, a 'Letöltés' link `download.php?id=<file_id>`-re mutat."

### 9.6.5. Elfelejtett jelszó

**Oldal neve:** `forgotpass.php`

**Cél:** Elfelejtett jelszó visszaállítása biztonsági kérdés válaszával. 2 lépésben működik: (1) felhasználónév + biztonsági válasz ellenőrzése rate limit-tel, (2) új jelszó megadása és mentése.

**Elérés / route:**

- URL: `.../forgotpass.php`
- Belépés szükséges? **Guest** (nem kell bejelentkezni)

**Bemenetek (inputok):**

- GET: nincs
- POST:
  - 1. lépés (biztonsági válasz ellenőrzés):
    - `forg-btn` (submit jelző)
    - `username` (string)
    - `security_answer` (string)
    - `csrf` (string)
  - 2. lépés (új jelszó beállítás):
    - `new-pass-btn` (submit jelző)
    - `password1` (string)
    - `password2` (string)
    - `csrf` (string)
- Session:
  - `csrf` (string): CSRF token
  - `pw_reset_user` (int): melyik user resetelhető
  - `pw_reset_ok` (bool): sikeres 1. lépés jelzője

**Folyamat (lépések):**

#### 1. Jogosultság ellenőrzés / session előkészítés

- `session_start()`
- CSRF token generálás, ha még nincs: `$_SESSION['csrf'] = bin2hex(random_bytes(32))`
- UI állapot változók: `showSecurityForm`, `showNewPassword`, `success`

## 2. Input validálás

- Minden POST esetén `check_csrf()` fut:
  - ha token nem egyezik: `exit('CSRF blocked')`
- 1. lépésben: `username` és `security_answer` nem lehet üres → `alert_redirect(error_all_fields_required)`
- 2. lépésben:
  - csak akkor engedett, ha `pw_reset_user` és `pw_reset_ok` be van állítva → különben `exit('Unauthorized')`
  - jelszavak egyezzenek → különben `alert_redirect(passwords_not_match)`
  - min. hossz: 8 → különben `alert_redirect(password_too_short)`

## 3. DB művelet(ek)

- **Rate limit / lock ellenőrzés** (1. lépés):
  - IP: `$_SERVER['REMOTE_ADDR']`
  - `password_reset_attempts` lekérdezés: `attempts`, `locked_until` adott (`username`, `ip`) párosra
  - ha `locked_until` a jövőben van → `alert_redirect(msg_too_many_attempts)`
- **User + biztonsági válasz ellenőrzés:**
  - `SELECT id, security_answer FROM users WHERE username = ? LIMIT 1`
  - ha nincs user → `alert_redirect(msg_user_not_found)`
  - `password_verify($answer, $user['security_answer'])`
    - **Helytelen válasz esetén:**
      - ha van rl rekord:
        - `attempts = attempts + 1`
        - ha `attempts >= MAX_ATTEMPTS (5)`:
          - `locked_until = now + 15 perc`
        - `UPDATE password_reset_attempts SET attempts=?, locked_until=?, last_attempt=NOW() ...`
      - ha nincs rl rekord:
        - `INSERT INTO password_reset_attempts (username, ip_address, attempts, last_attempt) VALUES (... , 1, NOW())`

- majd: `alert_redirect(msg_wrong_security_answer)`
- **Helyes válasz esetén:**
  - törli az rl rekordot: `DELETE FROM password_reset_attempts WHERE username=? AND ip_address=?`
  - beállítja sessionben:
    - `pw_reset_user = user.id`
    - `pw_reset_ok = true`
  - UI váltás: 2. lépés űrlap megjelenítése
- **Új jelszó mentése** (2. lépés):
  - hash: `password_hash($p1, PASSWORD_DEFAULT)`
  - `UPDATE users SET password = ? WHERE id = ? LIMIT 1`
  - `session_unset(); session_destroy();`
  - `success = true` (siker UI)

#### 4. Kimenet

- Siker:
  - "Jelszó sikeresen megváltoztatva" képernyő + link `reglog.php`
- 1. lépés UI:
  - felhasználónév + biztonsági válasz form (CSRF hidden fielddel)
- 2. lépés UI:
  - új jelszó + megerősítés form (CSRF hidden fielddel)
- Hiba:
  - `alert_redirect(...)` üzenetekkel (a pontos megvalósítás az `alert_redirect` függvénytől függ)

#### UI állapotok / felhasználói visszajelzés:

- Loading: nincs
- Empty state: nincs
- Error state:
  - Üres mezők → "minden mező kötelező"
  - User nem található
  - Rossz biztonsági válasz
  - Túl sok próbálkozás → lock (15 perc)
  - CSRF blokkolás → "CSRF blocked"
  - Jogosulatlan 2. lépés megnyitás → "Unauthorized"
  - Jelszavak nem egyeznek / túl rövid

- Success state:
  - Siker képernyő + "Ugrás a bejelentkezéshez" (`reglog.php`)

### Biztonság:

- SQL injection védelem: paraméterezett DB hívások (`db_query`, `db_exec`)
- XSS védelem:
  - oldalnyelv és CSRF érték escape-elve: `htmlspecialchars($lang)`, `htmlspecialchars($_SESSION['csrf'])`
  - a szövegek fordításból jönnek (`t()`), közvetlen user input nincs visszarendelve
- CSRF védelem: **van**, session token + `hash_equals`
- Rate limit / brute force védelem:
  - `password_reset_attempts` tábla `username` + `ip` alapon
  - MAX 5 próbálkozás után 15 perces lock (`locked_until`)
- Jelszó tárolás: `password_hash()` (bcrypt/argon a PHP defaulttól függően)
- Megjegyzés: a reset folyamat "biztonsági válasz" alapú, nincs email tokenes megerősítés ebben a flow-ban.

### Elfogadási kritériumok (tesztelhető):

- "CSRF token nélkül/hibás tokenel a kérés blokkolódik ('CSRF blocked')."
- "Üres `username` vagy `security_answer` esetén nem indul DB ellenőrzés/jelszó reset."
- "Nem létező user esetén 'user not found' üzenet jön (és nincs jelszó módosítás)."
- "5 rossz biztonsági válasz ugyanarra a username+IP párosra 15 perces lockot eredményez."
- "Helyes biztonsági válasz után megjelenik az új jelszó űrlap, és csak ekkor engedett a jelszócsere."
- "Nem egyező jelszavaknál nem történik DB update."
- "Sikeres jelszóváltáskor frissül a `users.password`, a session törlődik, és a siker képernyő jelenik meg, linkkel a `reglog.php` oldalra."
- "Jogosulatlanul (session jelzők nélkül) a 2. lépés POST nem fut le ('Unauthorized')."

## 9.6.6. Tanulócsoport

**Oldal neve:** `group.php`

**Cél:** Csoport adatlap megjelenítése és kezelése: leírás, privát/nyilvános állapot, tagság (csatlakozás/kilépés), tagok listája, (tulajnak) jelentkezések kezelése, csoport jegyzetek megjelenítése, (tagoknak) feltöltés, (tulajnak) jóváhagyás/elutasítás, valamint csoport törlése. Privát csoportnál a tartalom csak tagoknak/tulajnak látható.

**Elérés / route:**

- URL: `.../group.php?id=...` (az `id` paramétert a `group_init.php` tipikusan beolvassa)
- Belépés szükséges? **Vegyes**

- A csoport oldal megnyitható, de **privát csoportnál** a tartalom csak **tagoknak / tulajnak** elérhető.
- A műveletek (csatlakozás, kilépés, adminisztráció, feltöltés stb.) bejelentkezéshez kötöttek (ez a `group_init.php` / `group_actions.php` feladata).

### Bemenetek (inputok):

- GET:
  - `id` (int): csoport azonosító (a betöltéshez; közvetetten a `group_init.php` használja)
- POST (a gombok alapján, tényleges feldolgozás a `group_actions.php`-ban történik):
  - Tagság:
    - `join_group` (csatlakozás)
    - `kilepes` (kilépés)
  - Tag kezelés (tulaj):
    - `remove_member` + `remove_user_id` (tag eltávolítása)
    - `elfogadas` + `kezelt_user_id` (jelentkezés elfogadása)
    - `elutasitas` + `kezelt_user_id` (jelentkezés elutasítása)
  - Csoport jegyzet feltöltés (tag):
    - `uj_jegyzet`
    - `jegyzet_nev` (string)
    - `jegyzet_fajl` (file upload)
    - `jegyzet_leiras` (string, opcionális)
  - Csoport jegyzet moderálás (tulaj):
    - `jegyzet_elfogadas` + `jegyzet_id`
    - `jegyzet_elutasitas` + `jegyzet_id`
  - Csoport törlés (tulaj):
    - `csoport_torles`
- Egyéb (külön endpoint):
  - Csoport jelentése: `assets/php/report.php` felé POST
    - `type=group, target_id, redirect, reason`
- Session/Cookie:
  - Ebben a fájlban közvetlenül nincs cookie/session ellenőrzés, ezt a `group_init.php` és `group_actions.php` végzi (ott derül ki: ki az aktuális user, tag-e, tulaj-e).

### Folyamat (lépések):

## 1. Jogosultság / környezet inicializálás

- Biztonsági headerek beállítása.
- Include-ok:
  - `group_init.php` (csoport + user állapot betöltése: pl. `$csoport_id`, `$csoport_nev`, `$privat`, `$aktualis_felhasznalo_tag`, `$aktualis_felhasznalo_tulaj`, `$aktualis_felhasznalo_pending`, `$tulaj_id` stb.)
  - `group_actions.php` (POST műveletek feldolgozása: csatlakozás/kilépés/elfogadás/elutasítás/feltöltés/jóváhagyás/törlés...)
- Privát csoport gate:
  - ha `privat == 1` ÉS nem tag ÉS nem tulaj:
    - `$hiba_uzenet = "Ez egy privát csoport. A tartalom csak tagoknak látható."`
    - a tartalmi blokkok több helyen `$hiba_uzenet == ""` feltétellel vannak "lezárva"

## 2. Input validálás

- A megjelenítő fájlban főleg feltételes megjelenítés van.
- A tényleges validálás (pl. csatlakozás, feltöltés, jogosultság) a `group_actions.php`-ban történik.
- A report úrlapnál `redirect` mező `$_SERVER['REQUEST_URI']`-ből kerül kitöltésre és HTML-escape-elve.

## 3. DB művelet(ek)

- Tagok listája (csak tag/tulaj és nem privát tiltás esetén):
  - `SELECT group_members.*, users.username ... WHERE group_id=? AND status='accepted'`
- Függő jelentkezések (csak tulaj):
  - `SELECT group_members.*, users.username ... WHERE group_id=? AND status='pending'`
- Csoport jegyzetek (elfogadottak):
  - `SELECT group_files.*, users.username ... WHERE group_id=? AND is_approved=1 ORDER BY id DESC`
- Csoport jegyzetek (jóváhagyásra várók, csak tulaj):
  - `SELECT group_files.*, users.username ... WHERE group_id=? AND is_approved=0 ORDER BY id DESC`
- A tényleges írárok/törlések/jóváhagyások DB oldalon a `group_actions.php`-ban vannak.

## 4. Kimenet

- Siker: csoport oldal renderelése, állapottól függő blokkokkal:
  - leírás / privát jelzés
  - csatlakozás/kilépés gombok
  - taglista + (tulajnak) eltávolítás gomb
  - (tulajnak) pending kérelmek elfogadás/elutasítás
  - jegyzetek listája + (tagoknak) feltöltés
  - (tulajnak) várakozó jegyzetek elfogadás/elutasítás
  - (tulajnak) csoport törlése
- Hiba:
  - privát csoport esetén nem tag: figyelmeztetés szöveg, és a tartalom jelentős része rejtve marad

### UI állapotok / felhasználói visszajelzés:

- Loading: nincs
- Empty state:
  - nincs leírás: "Ehhez a csoporthoz még nincs leírás megadva."
  - nincs tag: "Még nincsenek tagok ebben a csoportban."
  - nincs pending: "Nincsenek függőben lévő jelentkezések."
  - nincs elfogadott jegyzet: "Még nincsenek elfogadott csoport jegyzetek."
  - nincs várakozó jegyzet (tulaj): "Nincs elfogadásra váró jegyzet."
- Error state:
  - privát csoport + nem tag: "Ez egy privát csoport. A tartalom csak tagoknak látható."
  - jelentés / törlés / elutasítás confirm ablakok (JS confirm)
- Success state:
  - a konkrét siker üzenetek/redirectek jellemzően a `group_actions.php`-ban vannak (pl. csatlakozás elfogadása, jegyzet jóváhagyása stb.)

### Biztonság:

- SQL injection védelem: paraméterezett lekérdezések (`db_query`)
- XSS védelem:
  - csoport név/leírás, tag nevek, jegyzet adatok escape-elve: `htmlspecialchars()`, leírásnál `nl2br(htmlspecialchars(...))`
  - report redirect mező `ENT_QUOTES`-szal escape-elve
- Fájl esetén:



- Feltöltés `multipart/form-data`-val történik, de a validálás/mentés nem itt van (várhatóan `group_actions.php` végzi: whitelist/max méret/path traversal védelem ott releváns)
- Megjelenítésnél a fájl elérési út így készül: `users/<username>/<file_name>` és direkt linkként szerepel (a tényleges biztonság attól függ, a feltöltésnél mennyire "safe" a fájlnev és a mappastruktúra)
- CSRF token: **nincs** a formokon (a POST akciók CSRF nélkül futnak, legalábbis ebben a fájlban)
- Jogosultság:
  - Privát csoport tartalom elrejtés UI-szinten itt történik (`$hiba_uzenet` gate)
  - A tényleges jogosultság-ellenőrzés (pl. ne tudj nem tulajként elfogadni/eltávolítani) a `group_actions.php` felelőssége

### Elfogadási kritériumok (tesztelhető):

- "Privát csoportban nem tag felhasználó nem látja a taglistát és a csoport jegyzeteket, és megkapja a privát figyelmeztetést."
- "Tag felhasználó látja az elfogadott taglistát és az elfogadott jegyzeteket."
- "Nem tag és nem pending felhasználó csatlakozás gombot lát; pending állapotban 'függőben' üzenetet lát."
- "Tulajdonos látja a pending jelentkezéseket és tud elfogadni/elutasítani."
- "Tulajdonos tud tagot eltávolítani (kivéve saját magát), és látja a csoport törlése gombot."
- "Tag tud új jegyzetet feltölteni a csoportba (űrlap megjelenik)."
- "Tulajdonos látja az elfogadásra váró jegyzeteket és tudja elfogadni/elutasítani."
- "A megjelenített dinamikus szövegek XSS-védetten jelennek meg (`htmlspecialchars`)."

### 9.6.7. Tanuló csoportok

**Oldal neve:** `groups.php`

**Cél:** A rendszerben elérhető tanuló csoportok listázása és megjelenítése, valamint belépési pont biztosítása új csoport létrehozásához (`create_group.php`) és egy adott csoport megnyitásához (`group.php?id=...`).

**Elérés / route:**

- URL: `.../groups.php`
- Belépés szükséges? **User** (cookie alapú azonosítás: `$_COOKIE['id']`)

**Bemenetek (inputok):**

- GET: nincs (a csoport megnyitása linkből megy tovább: `group.php?id=...`)
- POST: nincs
- Cookie:
  - `id` (string/int): bejelentkezett felhasználó azonosító (numerikus ellenőrzéssel)
- Session: nincs ebben a fájlban

## Folyamat (lépések):

### 1. Jogosultság ellenőrzés

- Biztonsági headerek beállítása.
- Ha nincs `$_COOKIE['id']` vagy nem numerikus (`ctype_digit`):
  - `redirect` → `reglog.php`
  - `exit`
- `$aktualis_felhasznalo_id = (int)$_COOKIE['id']` (a lapon érdeemben nem kerül felhasználásra, csak eltárolásra)

### 2. Input validálás

- Nincs klasszikus input (nincs GET/POST), csak a cookie `id` típusellenőrzése.

### 3. DB művelet(ek)

- Csoportok lekérése:
  - `SELECT * FROM groups ORDER BY id DESC`
  - Megjegyzés: itt közvetlen `$conn->query()` van használva (nincs paraméter, így nem injection-érzékeny ebben a formában).
- Eredmény bejárása és renderelése:
  - mezők: `id`, `name`, `description`, `is_private`

### 4. Kimenet

- Siker: rendereli a csoportkártyákat:
  - név (link: `group.php?id=...`)
  - privát/nyilvános badge
  - leírás vagy "nincs leírás"
  - "Csoport megnyitása" gomb
- Hiba (auth): redirect `reglog.php`
- Üres lista: "Nincs megjeleníthető csoport."

## UI állapotok / felhasználói visszajelzés:

- Loading: nincs
- Empty state:
  - Ha nincs egy csoport sem: "Nincs megjeleníthető csoport."
  - Egy adott csoportnál nincs leírás: "Ehhez a csoporthoz még nincs leírás megadva."
- Error state:

- Nem bejelentkezett user: redirect `reglog.php`
- Success state:
  - Csoportlista megjelenik kártyákban
  - CTA: "Új csoport létrehozása" → `create_group.php`

### Biztonság:

- SQL injection védelem:
  - A csoport lista query nem fogad inputot (fix SQL), ezért itt nincs közvetlen injection felület.
  - (Általánosan a projektben paraméterezett hívások javasoltak, de ennél a konkrét querynél nem kritikus.)
- XSS védelem:
  - Csoport név/leírás escape-elve: `htmlspecialchars()`, leírásnál `nl2br(htmlspecialchars(...))`
  - Privát/nyilvános státusz szöveg is escape-elve
- CSRF token: nem releváns (nincs állapotmódosító POST)
- Jogosultság:
  - Belépés cookie alapján kötelező (id numerikus ellenőrzéssel)

### Elfogadási kritériumok (tesztelhető):

- "Cookie `id` nélkül vagy nem numerikus `id`-vel a felhasználó `reglog.php`-ra redirectelődik."
- "Ha vannak csoportok a DB-ben, mind megjelenik a listában (id desc sorrendben)."
- "A csoport neve linkként a `group.php?id=<id>` oldalra vezet."
- "Privát csoportnál 'Privát csoport', nyilvánosnál 'Nyilvános csoport' címke látszik."
- "Ha egy csoportnak nincs leírása, a megfelelő empty szöveg jelenik meg."
- "A csoportnév és leírás XSS-védetten jelenik meg (`htmlspecialchars`)."

### 9.6.8. Főoldal

**Oldal neve:** `index.php`

**Cél:** Főoldal: köszöntő + névnap megjelenítése, legfrissebb feltöltések listázása, toplistas ("legjobbba értékelt") jegyzetek megjelenítése. Bejelentkezett felhasználónál kedvencek kezelése (toggle) és értékelés (1–5 csillag) közvetlenül a főoldalról.

### Elérés / route:

- URL: `.../index.php`
- Belépés szükséges? **Guest is elérheti**, de bizonyos funkciók **User**-hez kötöttek:
  - kedvenchez adás/törlés (POST)
  - értékelés (POST)

## Bemenetek (inputok):

- GET: nincs (a lap belső horgonyt használ: `index.php#file-<id>` redirect után)
- POST:
  - Kedvenc toggle:
    - `favorite-btn` (jelző)
    - `favorite_file_id` (int)
  - Értékelés:
    - `rate-btn` (jelző)
    - `rate_file_id` (int)
    - `rating` (int, 1–5)
- Cookie:
  - `id` (string/int): bejelentkezett user azonosítója (numerikus ellenőrzéssel)
- Session: nincs ebben a fájlban

## Folyamat (lépések):

### 1. Jogosultság ellenőrzés / user állapot felmérése

- Biztonsági headerek beállítása.
- `$isLoggedIn = isset($_COOKIE['id']) && ctype_digit($_COOKIE['id'])`
- Ha be van jelentkezve:
  - user lekérés: `SELECT id, username FROM users WHERE id = ? LIMIT 1`
  - értesítések száma: `SELECT id FROM notifys WHERE toid = ? AND readed = 0`
- Meghatározza a megjelenített nevet:
  - bejelentkezve: `username`, különben `t('guest')`

### 2. Névnap lekérdezés

- `$today = date("m-d")`
- `SELECT nevek FROM namedays WHERE datum = ? LIMIT 1`
- ha nincs találat: `t('nameday_none_today')`

### 3. Lista lekérdezések (főoldali tartalom)

- "Top rated" (oldalsáv):
  - `files + ratings LEFT JOIN`
  - `AVG(r.rating)` és `COUNT(r.id)`
  - rendezés: `avg_rating DESC`, majd `rating_count DESC`

- limit 8
- "Legfrissebb feltöltések":
  - `SELECT * FROM files ORDER BY id DESC LIMIT 12`

#### 4. Kedvencek kezelése (POST favorite-btn)

1. Ha nincs bejelentkezve: redirect → `reglog.php`
2. `favorite_file_id` intre castolva
3. Ha `file_id > 0`:
  - ellenőrzés: `SELECT id FROM favorites WHERE file_id=? AND user_id=? LIMIT 1`
  - ha létezik → `DELETE FROM favorites ...`
  - ha nem létezik → `INSERT INTO favorites (file_id, user_id) VALUES (?, ?)`
4. redirect → `index.php#file-<file_id> + exit`

#### 5. Értékelés kezelése (POST rate-btn)

1. Ha nincs bejelentkezve: redirect → `reglog.php`
2. `rate_file_id, rating` intre castolva
3. Validálás: `file_id > 0` és `rating 1..5`
4. Ellenőrzés: `SELECT id FROM ratings WHERE file_id=? AND user_id=? LIMIT 1`
  - ha van → `UPDATE ratings SET rating=? ...`
  - ha nincs → `INSERT INTO ratings (file_id, user_id, rating) VALUES (?, ?, ?)`
5. redirect → `index.php#file-<file_id> + exit`

#### 6. Kimenet (render)

- Hero rész: köszöntés névvel + névnap + "Feltöltés" gomb (bejelentkezve `upload.php`, különben `reglog.php`)
- "Új feltöltések" rács:
  - kártyák fájlként, uploader lekéréssel (`users.username`)
  - rating átlag+db kiírás (külön query fájlként)
  - kedvenc gomb:
    - bejelentkezve: POST toggle
    - vendég: link `reglog.php`
  - értékelés:

- bejelentkezve: radio csillagok (onchange submit), és "Te: X/5"
- vendég: "Értékeléshez jelentkezz be."

- Oldalsáv: top rated lista linkekkel `note.php?id=...`

### UI állapotok / felhasználói visszajelzés:

- Loading: nincs
- Empty state:
  - kódban nincs külön "nincs találat" UI a friss/top listákra (csak `if ($latest_result && $latest_result->num_rows > 0)` jellegű csendes elrejtés)
- Error state:
  - bejelentkezés nélkül POST művelet → redirect `reglog.php`
  - hibás rating (nem 1–5) vagy `file_id <= 0`: nincs üzenet, csak redirect vissza az anchorra
- Success state:
  - kedvenc toggle és értékelés után: redirect `index.php#file-<id>` (vizuálisan: kedvenc gomb állapota / csillag kijelölés változik)

### Biztonság:

- SQL injection védelem:
  - Paraméterezett lekérdezések `db_query` / `db_exec` használatával a user inputot érintő részekben (favorites, ratings, user, nameday, uploader).
  - A "top rated" és "latest" lekérdezések fix SQL-lel mennek (`$conn->query`), input nélkül.
- XSS védelem:
  - Megjelenített dinamikus mezők escape-elve: `htmlspecialchars($displayName)`, fájlnev, uploader név, névnap stb.
- CSRF token: **nincs**
  - A kedvenc/értékelés POST-ok CSRF védelem nélkül futnak.
- Fájl esetén:
  - Itt nincs feltöltés; letöltés külön endpoint (`assets/php/download.php?id=...`)
- Egyéb megjegyzés (teljesítmény):
  - Listázásnál több "N+1" query van (uploader + avg rating + user rating + favorite státusz fájlanként).

### Elfogadási kritériumok (tesztelhető):

- "Vendég felhasználó látja a főoldalt, de kedvenchez adás/értékelés POST esetén `reglog.php`-ra redirectelődik."
- "Bejelentkezett usernél a köszöntés a username-et mutatja, és az értesítések száma lekérhető (navbarhoz)."
- "A legfrissebb 12 fájl megjelenik (id szerinti csökkenő sorrendben)."
- "Kedvenc gomb megnyomására a `favorites` táblában a rekord létrejön vagy törlődik (toggle), majd redirect `index.php#file-<id>`."
- "Értékelés 1–5 tartományban mentésre kerül (insert vagy update), majd redirect `index.php#file-<id>`."
- "Hibás rating vagy hibás file\_id esetén nem történik DB írás."
- "A megjelenített szövegek XSS-védetten jelennek meg (`htmlspecialchars`)."

### 9.6.9. Feltöltés

**Oldal neve:** `upload.php`

#### Cél:

Az oldal célja, hogy a bejelentkezett felhasználók új jegyzeteket tölthessenek fel a rendszerbe. A feltöltés során a fájl mellett metaadatok (név, leírás, kategória) kerülnek rögzítésre, melyek alapján a jegyzet később kereshető és listázható.

#### Elérés / route:

- URL: `.../upload.php`
- Belépés szükséges? **User**

Nem bejelentkezett felhasználók automatikusan a bejelentkezési oldalra kerülnek átirányításra.

#### Bemenetek (inputok):

- GET: nincs
- POST:
  - `name` – a jegyzet megjelenített neve
  - `description` – opcionális szöveges leírás
  - `subject` – kategória / tantárgy
  - `file` – feltöltendő fájl
- Session / Cookie:
  - `id` – bejelentkezett felhasználó azonosítója

#### Folyamat (lépések):

1. Jogosultság ellenőrzés:  
Ellenőrzi, hogy létezik-e érvényes bejelentkezett felhasználó. Ha nem, redirect történik.
2. Input validálás:  
Kötelező mezők ellenőrzése, fájl meglétének vizsgálata, fájlnev és méret validálása.

### 3. DB műveletek:

A fájl elmentése a felhasználó saját könyvtárába, majd rekord beszúrása a `files` táblába `db_prepared()` használatával.

### 4. Kimenet:

- siker: redirect a főoldalra vagy a feltöltött jegyzet oldalára
- hiba: JavaScript alert formájában megjelenített hiba

### UI állapotok / felhasználói visszajelzés:

- Loading: nincs explicit
- Empty state: nem értelmezett
- Error state: hiányzó mezők, tiltott fájl
- Success state: "Sikeres feltöltés" üzenet + redirect

### Biztonság:

- SQL injection védelem: `db_prepared()`
- XSS védelem: megjelenítésnél `htmlspecialchars()`
- Fájlkezelés: whitelistelt kiterjesztések, könyvtár-szintű szeparáció
- CSRF token: nincs (fejlesztési kompromisszum)

### Elfogadási kritériumok:

- Hibás vagy hiányos input esetén nem történik adatbázis-írás
- Sikeres feltöltés után pontos redirect történik
- Nem bejelentkezett felhasználó nem érheti el az oldalt

## 9.6.10. Kereső

**Oldal neve:** `search.php`

### Cél:

A rendszerben elérhető jegyzetek keresése és listázása kulcsszó, rendezési szempont és lapozás alapján.

### Elérés / route:

- URL: `.../search.php`
- Belépés szükséges? **Guest**

### Bemenetek (inputok):

- GET:
  - `q` – keresőkifejezés
  - `sort` – rendezési mód
  - `page` – lapozás
- POST: nincs
- Session: nincs



### Folyamat (lépések):

1. Jogosultság ellenőrzés: nem szükséges.
2. Input validálás:  
A keresőkifejezés megtisztítása és hosszellenőrzése.
3. DB művelet:  
Paraméterezett SQL lekérdezés a **files** táblára.
4. Kimenet:
  - siker: találatok listázása
  - hiba: "Nincs találat" állapot

### UI állapotok / felhasználói visszajelzés:

- Loading: nincs
- Empty state: nincs találat
- Error state: nincs
- Success state: találati lista

### Biztonság:

- SQL injection védelem
- XSS védelem
- CSRF token: nem releváns

### Elfogadási kritériumok:

- Keresés nem okoz SQL hibát
- Üres keresés is stabilan működik
- Találatok helyesen jelennek meg

### 9.6.11. Jegyzet

**Oldal neve:** **note.php**

#### Cél:

Egy adott jegyzet részletes megjelenítése, beleértve a letöltést, értékeléseket és kapcsolódó információkat.

#### Elérés / route:

- URL: **.../note.php?id=**
- Belépés szükséges? **Guest**

#### Bemenetek (inputok):

- GET:
  - **id** – jegyzet azonosító

- POST:
  - értékelés, kedvencek
  - Session / Cookie:
  - `id` (opcionális)

#### Folyamat (lépések):

1. ID validálás (numerikus, létező).
2. Jegyzet adatainak lekérdezése.
3. Kapcsolódó adatok betöltése (feltöltő, rating).
4. Oldal renderelése.

#### UI állapotok:

- Empty state: jegyzet nem található
- Error state: hibás ID
- Success state: teljes jegyzetoldal

#### Biztonság:

- SQL injection védelem
- XSS védelem
- Letöltés külön endpointon

#### Elfogadási kritériumok:

- Nem létező jegyzet nem jelenik meg
- Jogosultság nélküli művelet nem hajtható végre

### 9.6.12. Jegyzet statisztikák

Oldal neve: `note_stats.php`

#### Cél:

Egy jegyzet statisztikai adatainak megjelenítése a feltöltő számára.

#### Elérés / route:

- URL: `.../note_stats.php?id=`
- Belépés szükséges? **User (tulajdonos)**

#### Bemenetek (inputok):

- GET:
  - `id`
- Session / Cookie:
  - `id`

**Folyamat (lépések):**

1. Jogosultság ellenőrzés (csak tulajdonos).
2. ID validálás.
3. Aggregált statisztikák lekérdezése.
4. Megjelenítés.

**Biztonság:**

- SQL injection védelem
- XSS védelem

**Elfogadási kritériumok:**

- Más felhasználó nem fér hozzá
- Statisztikák helyesek

**9.6.13. Profil**

**Oldal neve:** `profile.php`

**Cél:**

Felhasználói profil nyilvános megjelenítése jegyzetekkel és jelvényekkel.

**Elérés / route:**

- URL: `.../profile.php?userid=`
- Belépés szükséges? **Guest**

**Bemenetek (inputok):**

- GET:
  - `userid`
- Session:
  - `id` (opcionális)

**Folyamat (lépések):**

1. User ID validálás.
2. Profiladatok lekérdezése.
3. Kapcsolódó jegyzetek és badge-ek betöltése.
4. Oldal renderelése.

**Biztonság:**

- SQL injection védelem
- XSS védelem

**Elfogadási kritériumok:**

- Nem létező profil nem jelenik meg
- Adatok helyesen jelennek meg

#### 9.6.14. Üzenetek

**Oldal neve:** `messages.php`

**Cél:**

Privát üzenetküldés és beszélgetések kezelése felhasználók között.

**Elérés / route:**

- URL: `.../messages.php`
- Belépés szükséges? **User**

**Bemenetek (inputok):**

- GET:
  - `with`
- POST:
  - `message`
- Session / Cookie:
  - `id`

**Folyamat (lépések):**

1. Jogosultság ellenőrzés.
2. Üzenet validálás.
3. Mentés adatbázisba.
4. Beszélgetés frissítése.

**Biztonság:**

- SQL injection védelem
- XSS védelem

**Elfogadási kritériumok:**

- Nem bejelentkezett user nem írhat
- Üzenet mentésre kerül

#### 9.6.15. Értesítések

**Oldal neve:** `notify.php`

**Cél:**

Felhasználói értesítések megjelenítése és olvasott státusz kezelése.

**Elérés / route:**

- URL: `.../notify.php`
- Belépés szükséges? **User**

**Bemenetek (inputok):**

- GET:
  - `read`
- Session / Cookie:
  - `id`

**Folyamat (lépések):**

1. Jogosultság ellenőrzés.
2. Értesítések lekérdezése.
3. Státusz frissítése.
4. Lista megjelenítése.

**9.6.16. Bejelentkezés és Regisztráció**

**Oldal neve:** `reglog.php`

**Cél:** Regisztráció és bejelentkezés kezelése.

**Elérés / route:**

- URL: `.../reglog.php`
- Belépés szükséges? **Guest**

**Bemenetek (inputok):**

- POST:
  - `username`
  - `password`
  - regisztrációs adatok
- Session: több lépcsős auth

**Folyamat (lépések):**

1. Input validálás.
2. Felhasználó ellenőrzés / létrehozás.
3. Session indítás.
4. Redirect főoldalra.

**Biztonság:**

- SQL injection védelem

- XSS védelem
- Jelszó hash

#### Elfogadási kritériumok:

- "Hibás jelszó nem enged be"
- "Sikeres login redirectel"

#### 9.6.17. Email-s visszaigazolás

**Oldal neve:** `reg-ver.php`

**Cél:** Regisztrációs e-mail vagy kód megerősítése.

#### Elérés / route:

- URL: `.../reg-ver.php`
- Belépés szükséges? **Guest**

#### Bemenetek (inputok):

- GET:
  - `code`
- Session: nincs

#### Folyamat (lépések):

1. Kód validálás.
2. Felhasználó aktiválása.
3. Redirect loginra.

#### Biztonság:

- SQL injection védelem
- XSS védelem

#### Elfogadási kritériumok:

- "Hibás kód nem aktivál"
- "Sikeres aktiválás után login"

#### 9.6.18. Adatvédelmi tájékoztató

**Oldal neve:** `privacy.php`

**Cél:** Adatvédelmi tájékoztató megjelenítése.

#### Elérés / route:

- URL: `.../privacy.php`
- Belépés szükséges? **Guest**

### Bemenetek (inputok):

- GET: nincs
- POST: nincs
- Session: nincs

### Folyamat (lépések):

1. Statikus tartalom betöltése.
2. Oldal renderelése.

### UI állapotok:

- Empty state: nincs
- Success state: tartalom megjelenik

### Biztonság:

- XSS védelem (statikus tartalom)

### Elfogadási kritériumok:

- "Az oldal bárki számára elérhető"
- "A tartalom helyesen jelenik meg"

## 9.7 Security checklist (dev)

### Security checklist

- Minden DB lekérdezés: `db_prepared()`
- Output escaping: HTML-ben `htmlspecialchars()`
- File upload:
  - MIME ellenőrzés
  - whitelist kiterjesztés
  - max size limit
  - fájlnev randomizálás / path traversal védelem
- Session:
  - session fixation elleni védelem (login után `session_regenerate_id`)
- CSRF (ha nincs): formokhoz token javasolt

## 9.8 Debug / logging

### Logging / debug

- PHP error log: XAMPP Apache/PHP log
- App log: `assets/logs/` (ha használjátok)
- Tipikus debug lépések:
  - `display_errors` devben (productionben OFF)
  - SQL hibák: `mysqli_error` / wrapper logolás

## 9.9. Változásnapló (CHANGELOG)

A projekt hivatalos változásnaplója a `docs/CHANGELOG.md` fájlban található.

Minden érdemi módosítást itt kell rögzíteni, hogy követhető legyen a fejlesztés és a verziók közötti különbség.

### Szabályok

- **A legújabb verzió mindig felül legyen.**
- **Minden módosítást rögzíteni kell** (feature, bugfix, refaktor, eltávolítás, security patch).
- A bejegyzéseket az alábbi kategóriák egyikébe kell sorolni:
  - **Added** – új funkció
  - **Changed** – módosítás meglévő funkción
  - **Fixed** – hibajavítás
  - **Removed** – eltávolított funkció/elem
  - **Security** – biztonsági frissítés
- **Dátum formátum:** `YYYY-MM-DD`
- **Verziózás:** `major.minor.patch` (pl. `1.0.3`)
- **Szerző megjelölése kötelező** a bejegyzés végén (`Changelog by: Név`)

### Mikor frissítjük?

- **Minden merge / release után** frissíteni kell.
- Ha több kisebb commitból áll egy változás, akkor a changelogba elég a **összefoglaló** jellegű leírás.

### Formátum

```
[1.X.X] - 2025-0X-0X
Added
• ...
Changed
• ...
Fixed
• ...
Removed
• ...
Security
• ...
Changelog by: Neved
```

### Példa bejegyzés (mintaként)



[1.2.0] - 2026-01-14

Added

- Admin felületen külön lista és jóváhagyási folyamat az egyedi profil CSS kérelmekhez.

Changed

- Fordításkezelés átalakítva adatbázis-alapú ``t()`` rendszerre, automatikus missing-key seedinggel.

Fixed

- Fordítás dump import: duplikált ``translations`` sorok kezelése és biztonságos import sorrend javaslatok.

Security

- Lekérdezések egységesítése ``db_prepared()`` wrapperrel az SQL injection kockázat csökkentésére.

Changelog by: Csontos Kincső Anasztázia

## 9.10. Dokumentáció karbantartás

A projekt dokumentációja a [docs/dokumentáció.md](#) fájlban található.

### Mikor frissítjük?

- Minden nagyobb funkció (feature) hozzáadása után
- Adatbázis módosítás (új tábla / mező / index) után
- Biztonsági változtatás vagy refaktor után
- Release / merge után a [docs/CHANGELOG.md](#) frissítése kötelező

### Mit kell ilyenkor frissíteni?

- Tartalomjegyzék (ha új fejezet került be)
- Érintett fejezet(ek) leírása (pl. Backend / Frontend / Deployment)
- DB séma (ha változott)
- Changelog bejegyzés (verzió + dátum + kategóriák + szerző)

### Minimum elv

Ha nincs idő mindent átírni, legalább:

1. Changelog + 2) érintett fő folyamat rövid leírása kerüljön be.

Megjegyzés: A változásnapló írásának szabályai és formátuma a **8.9. Változásnapló (Changelog)** fejezetben található.

## 10. API Dokumentáció

## 11. Jövőbeli Tervek

### 11.1. Kollaboratív tanulási eszközök

A Jegyzetár közösségi aspektusának bővítése érdekében a cél, hogy a felhasználók együtt dolgozhassanak jegyzeteken és tanulási anyagokon.

#### Lehetséges fejlesztések:

- **Valós idejű jegyzet szerkesztés:** Több felhasználó egyszerre szerkeszthet egy dokumentumot, hasonlóan a Google Docs-hoz.
- **Csoportos tanulási szobák:** Felhasználók létrehozhatnak virtuális tanulócsoportokat, ahol fájlok megosztása, chat és feladatok követése egyszerre történhet.
- **Komment és annotáció rendszer:** A jegyzeteken belül jelölhetnek részeket, és hozzáfűzhetnek személyes megjegyzéseket vagy kérdéseket.
- **Közös naptár és feladatlista:** Tanulócsoportok eseményeket, határidőket és feladatokat oszthatnak meg egymással.

### 11.2. API és harmadik fél integrációk

A Jegyzetár platformot érdemes külső szolgáltatásokkal összekapcsolni, hogy a felhasználók minél több eszközt használhassanak egy helyen.

#### Lehetséges fejlesztések:

- **OAuth integrációk:** Google, Microsoft, Discord vagy EduID fiókokkal történő bejelentkezés támogatása.
- **Fájlmegosztó szolgáltatások integrálása:** Dropbox, Google Drive, OneDrive feltöltés és letöltés közvetlenül a Jegyzetárból.
- **Tanulási eszközök API kapcsolatai:** Kahoot, Quizlet vagy egyéb oktatási platformokkal való integráció a könnyebb feladat és teszt megosztás érdekében.
- **Push értesítések külső csatornákra:** Discord bot vagy email értesítések, ha valaki új jegyzetet tölt fel, vagy kommentel a fájlodra.

### 11.3. Big data analitika

A platform folyamatos adatgyűjtésével és elemzésével javítható a felhasználói élmény és a tartalom minősége.

#### Lehetséges fejlesztések:

- **Felhasználói aktivitás elemzése:** Kinek milyen típusú jegyzetek tetszenek, mennyi időt töltenek az oldalon, mely tantárgyak a legnépszerűbbek.
- **Tartalom minőség értékelése:** Legjobbra értékelt és leggyakrabban letöltött anyagok kiemelése.
- **Tanulási szokások feltérképezése:** Időszakos jelentések készítése, hogy mikor aktívak a diákok, milyen tantárgyakhoz kell több támogatás.
- **Prediktív ajánlások:** A felhasználók korábbi aktivitása alapján automatikus jegyzet- és csoport-ajánlások.
- **Admin riportok:** Statisztikák a feltöltött tartalmakról, felhasználói aktivitásról és a csoportok működéséről a hatékonyabb menedzsmenthez.

### 11.4. AI-alapú keresés és javaslatok

A Jegyzetár fejlesztése során gépi tanulási és mesterséges intelligencia (AI) megoldások is bevezetésre kerülnek, hogy a felhasználók gyorsabban és hatékonyabban találják meg a számukra releváns tartalmakat.

Főbb funkciók:

- **Személyre szabott ajánlások:** Az AI elemzi a felhasználó korábbi kereséseit, letöltéseit és érdeklődési köreit, majd ezek alapján ajánl új jegyzeteket vagy segédanyagokat.
- **Intelligens kereső:** A keresési találatok sorrendjét a rendszer a felhasználó szokásaihoz és preferenciáihoz igazítja.
- **Tartalom szűrés és kategorizálás:** Az AI segít a feltöltött anyagok automatikus címkézésében és kategorizálásában.
- **Tanulási útvonalak ajánlása:** A rendszer javaslatokat adhat, hogy milyen anyagokat érdemes elolvasni egy adott témakörben.

**Előnyök:**

- Gyorsabb és pontosabb keresés
- Személyre szabott tanulási élmény
- Nagyobb felhasználói elégedettség

## 11.5. Gamifikáció

A felhasználói aktivitás ösztönzése érdekében a Jegyzetár gamifikációs elemeket vezet be, amelyek játékosabbá és motiválóbbá teszik a platform használatát.

**Főbb funkciók:**

- **Pontgyűjtés:** A felhasználók pontokat szerezhetnek különböző tevékenységekért, például regisztrációért, jegyzetfeltöltésért, hozzászólásért vagy értékelésért.
- **Jelvények és rangok:** Különböző mérföldkövek eléréséért (pl. első feltöltés, 100. letöltés) digitális jelvényeket és rangokat kapnak a felhasználók.
- **Ranglisták:** A legaktívabb vagy legnépszerűbb felhasználók megjelennek a közösségi ranglistákon.
- **Kihívások és küldetések:** Időszakos vagy tematikus kihívások teljesítésével további jutalmak szerezhetők.

**Előnyök:**

- Felhasználói elköteleződés növelése
- Közösségi aktivitás ösztönzése
- Pozitív visszacsatolás a felhasználók számára

## 11.6. Mobil Applikáció

A Jegyzetár jövőbeli fejlesztési tervei között szerepel egy natív mobilalkalmazás elkészítése Android és iOS platformokra. A mobil app célja, hogy a felhasználók még kényelmesebben érhessek el a jegyzeteket, tölthessenek fel fájlokat, és kommunikálhassanak egymással útközben is.

**Főbb funkciók:**

- Jegyzetek böngészése, letöltése és feltöltése közvetlenül a mobilról
- Push értesítések új üzenetekről, barátkérésekről, kommentekről
- Felhasználói profil szerkesztése, profilkép módosítása
- Barátok kezelése, üzenetküldés
- Offline elérés a letöltött jegyzetekhez

#### Technológiai lehetőségek:

- **React Native** vagy **Flutter** a multiplatform fejlesztéshez
- REST API integráció a meglévő backenddel
- Biztonságos bejelentkezés és adatkezelés

### 11.7. Interaktív tesztek

A Jegyzetár jövőbeli fejlesztései között kiemelt szerepet kapnak az **interaktív tesztek**, amelyek célja a felhasználók hatékony felkészítése a szakmai vizsgákra, különösen az **interaktív (gyakorlati) vizsgarészre**, valamint kiegészítő jelleggel az **írásbeli vizsgára** is.

Az interaktív teszt modul célja, hogy a tanulók ne csak passzívan olvassák a jegyzeteket, hanem **aktívan gyakorolhassák** a vizsgán előforduló feladattípusokat.

#### Tervezett funkciók:

- **Interaktív kérdéssorok:**

Többféle feladattípus támogatása, például:

- feleletválasztós kérdések,
- igaz-hamis állítások,
- párosító feladatok,
- kódrészlet-elemzéshez kapcsolódó kérdések.

- **Azonnali visszajelzés:**

A felhasználók a válaszadás után rögtön visszajelzést kapnak:

- helyes / helytelen válasz jelzése,
- rövid magyarázat vagy hivatkozás a kapcsolódó jegyzetre.

- **Vizsgaszimulációs mód:**

Időre kitölthető tesztek, amelyek a valódi interaktív vizsga felépítését és időkorlátait modellezzik.

- **Témakörönkénti gyakorlás:**

A tesztek tantárgyakhoz vagy témakörökhöz kapcsolhatók (pl. adatbázis-kezelés, webfejlesztés, algoritmusok), így célzott felkészülést tesznek lehetővé.

- **Eredmények nyomon követése:**

A rendszer eltárolja a korábbi teszteredményeket, amelyek alapján:

- megjeleníthető a fejlődés,
- azonosíthatók a gyengébb területek.

#### Kapcsolódás az írásbeli vizsgához:

Az interaktív tesztek mellett külön aloldalon elérhetők lennének az **írásbeli vizsgához kapcsolódó feladatgyűjtemények**, PDF formátumban. Ezek:

- letölthetők és offline is használhatók,
- tartalmazhatják korábbi évek feladatait,
- kiegészíthetik az interaktív gyakorlást elméleti jellegű feladatokkal.

### **Előnyök:**

- Aktív tanulás és gyakorlás támogatása
- Vizsgahelyzethez hasonló környezet biztosítása
- Egyéni haladás és fejlődés követése
- A jegyzetek gyakorlati alkalmazásának elősegítése

## **12. Ki mit készített?**

A projekt fejlesztése során a csapattagok eltérő területekért feleltek, ugyanakkor több esetben együttműködés is történt. Az alábbi bontás a fő felelősségi köröket és elkészült funkciókat foglalja össze.

### **12.1. Baranyi Norbert**

#### **Hitelesítés, jegyzetfunkciók és adatkezelés**

- Kétlépcsős hitelesítés (2FA) megvalósítása
  - e-mail alapú kódgenerálás és ellenőrzés
  - `2fa.php`, `2fa_codes` tábla
- 2FA-hoz kapcsolódó profil funkciók (be- és kikapcsolás)
- Regisztrációs e-mail visszaigazoló rendszer
  - token alapú aktiváció (`reg-ver.php`, tokens)
- Jegyzet részletező oldal fejlesztése (`note.php`)
  - értékelési rendszer
  - kedvencekhez adás
- Kedvencek mentése és kezelése
  - adatbázis logika
  - kedvencek megjelenítése
- Jegyzet tagek kezelése
  - tagek adatbázisban
  - tagek hozzárendelése jegyzetekhez
  - tagek megjelenítése kereséskor
- Jegyzet tagelés továbbfejlesztése (pl. középiskola, egyetem)

- „Ugrás jegyzetre” funkció megvalósítása
- Adatbázis struktúra bővítése
  - tags tábla
- Feltöltési logika javítása (`upload.php`)

## 12.2. Csontos Kincső Anasztázia

### UI/UX, biztonság, profil és rendszer szintű fejlesztések

- Teljes felület újradizájnolása
  - Aurora UI stílus
  - reszponzív layout
- Navbar újratervezése
- Oldal redesign és vizuális egységesítés
- UI finomhangolások
  - túlzott gradiensek csökkentése
  - kártyák, gombok igazítása
- Multilanguage rendszer
  - `lang.php`
  - fordítások adatbázisból
- Profil oldal fejlesztése
  - profiladatok szerkesztése
  - bemutatkozás karakterkorlátozása
  - egyedi CSS kérelmek
  - jelvények (badge-ek) megjelenítése
  - custom profil design
- Más felhasználó adatainak szerkesztése (JavaScript alapú megoldás)
- Biztonsági fejlesztések
  - adatbázis helper függvények (`db_query`, `db_stmt`, `db_prepared`)
  - SQL injection védelem
  - adatbázis-módosítások védelme
  - biztonsági kérdések hash-elése
- Jelentés / report rendszer
  - jelentés gombok
  - admin oldali kezelés

- Jegyzet statisztika megjelenítése
- Értékelési rendszer hibajavításai (főoldalon nem működő értékelés)
- Discord OAuth alapú bejelentkezés
- Születésnap „meglepetés” funkció
- Üzenetek UI átdolgozása
- Dokumentáció elkészítése

## 12.3. Szekeres Levente

### Tanulócsoportok, hirdetések és mobil optimalizáció

- Tanulócsoport funkciók teljes implementációja
  - `groups.php`
  - `group.php`
  - `create_group.php`
  - `group_init.php`
- Csoporttagság kezelése
  - tulajdonos
  - elfogadott tagok
  - függőben lévő jelentkezések
- Csoporton belüli jegyzetfeltöltés
  - jóváhagyási rendszer
  - moderáció
- Tanulócsoportos funkciók hibajavítása és stabilizálása
- Hirdetések megjelenítése az oldalon
- Mobil navigáció javítása (mobil nav fix)
- UI/Design fejlesztések a csoportos oldalaknál

Íme egy **dokumentáció-kompatibilis**, hivatalos hangvétellű megfogalmazás, amit **szó szerint be tudsz illeszteni**. Úgy írtam meg, hogy illeszkedjen a 11. fejezet stílusához és logikájához.

### Javasolt megoldás

Érdemes **külön alpontként** szerepeltetni, mert ez **külön platform (C# WinForms)** és **közös munka** volt Szekeres Levente és Baranyi Norbert részéről.

## 12.4. Szekeres Levente & Baranyi Norbert

## Offline C# WinForms asztali alkalmazás – jegyzetnézegető koncepció

- Offline működésre tervezett C# alapú asztali alkalmazás fejlesztése **WinForms** technológiával
- Az alkalmazás célja egy **offline jegyzetnézegető rendszer** létrehozása volt
- Tervezett működés:
  - online állapotban a jegyzetek **lokális letöltése és tárolása**
  - a letöltött jegyzetek **internetkapcsolat nélkül is megtekinthetők**
- A megoldás lehetővé teszi a tananyagok elérését olyan környezetben is, ahol nincs folyamatos internetkapcsolat
- A projekt célja az volt, hogy a webes rendszer kiegészítéseként egy **offline tanulást támogató kliensalkalmazás** jöjjön létre

## Összegzés

A fejlesztés során:

- **Baranyi Norbert** elsősorban az **auth, jegyzetkezelés és adatstruktúrák** megvalósításáért felelt.
- **Csontos Kincső Anasztázia** a **felhasználói élményért, biztonságért, profil- és rendszerfunkciókért**, valamint a dokumentációért.
- **Szekeres Levente** a **tanulócsoporthoz funkciókért, mobil optimalizációért és hirdetésekért** volt felelős.
- **Szekeres Levente és Baranyi Norbert** közösen egy **C# WinForms alapú offline jegyzetnézegető asztali alkalmazás** koncepcióját és alapjait valósították meg.

A projekt a feladatok megosztásával és együttműködéssel valósult meg.

## 13. Licenz

Adatkezelési és tartalomfeltöltési irányelvek (röviden)

- A felhasználó **nem tölthet fel** személyes adatokat tartalmazó dokumentumot (pl. lakcím, telefonszám, igazolvány, osztálynapló).
- A felhasználó vállalja, hogy a feltöltött anyag **nem sért szerzői jogot**, vagy rendelkezik megosztási joggal.
- A rendszer fenntartja a jogot szabálytalan tartalmak **eltávolítására**, és a felhasználói fiók korlátozására (admin moderáció).

Ez a projekt saját projektmunkás licenz alatt áll. A forráskód és a dokumentáció kizárólag oktatási célokra használható fel, kereskedelmi felhasználásra nem engedélyezett.

A felhasználók vállalják, hogy nem töltenek fel jogvédett tartalmat.